

miniprogram/app.ts

```
0001 import { runtimeConfig } from './config/runtime';
0002
0003 interface IAppOption {
0004   globalData: {
0005     apiBaseUrl: string;
0006     cloudbaseEnvId: string;
0007     environmentKind: 'demo' | 'authorized' | 'unconfigured';
0008   };
0009 }
0010
0011 App<IAppOption>({
0012   globalData: {
0013     // WeChat external config carries only non-secret build values.
0014     apiBaseUrl: runtimeConfig.apiBaseUrl,
0015     cloudbaseEnvId: runtimeConfig.cloudbaseEnvId,
0016     environmentKind: runtimeConfig.environmentKind,
0017   },
0018   onLaunch() {
0019     // 初始化阶段不发起网络请求；环境状态由后续服务端配置确认。
0020   },
0021 });
```

admin/src/main.ts

```
0001 import { createApp } from 'vue';
0002 import './styles/tokens.css';
0003 import './styles/base.css';
0004 import App from './App.vue';
0005 import router from './router';
0006 import { createPinia } from 'pinia';
0007
0008 createApp(App).use(createPinia()).use(router).mount('#app');
```

admin/src/App.vue

```
0001 <script setup lang="ts">
0002 import { computed, onMounted, onUnmounted, ref } from 'vue';
0003 import DesktopWidthNotice from '@components/DesktopWidthNotice.vue';
0004
0005 const width = ref(typeof window === 'undefined' ? 1440 : window.innerWidth);
0006 const blocked = computed(() => width.value < 1280);
0007 const updateWidth = () => {
0008   width.value = window.innerWidth;
0009 };
0010 onMounted(() => window.addEventListener('resize', updateWidth));
0011 onUnmounted(() => window.removeEventListener('resize', updateWidth));
0012 </script>
0013
0014 <template>
0015   <DesktopWidthNotice v-if="blocked" :viewport-width="width" />
0016   <router-view v-else />
0017 </template>
```

backend/app/main.py

```
0001 """FastAPI application factory and dependency-free health endpoint."""
0002
0003 from typing import Any
0004
0005 from fastapi import FastAPI, Request
0006 from pydantic import BaseModel
0007
0008 from app.api.admin_auth import router as admin_auth_router
0009 from app.api.admin_identity import router as admin_identity_router
0010 from app.api.admin_workbench import router as admin_workbench_router
0011 from app.api.assessment import contract_router as assessment_contract_router
0012 from app.api.assessment import router as assessment_router
0013 from app.api.auth import me_router
0014 from app.api.auth import router as auth_router
0015 from app.api.dependencies import (
0016     _error_response,
```

backend/app/main.py

```
0017     register_exception_handlers,
0018     resolve_request_id,
0019 )
0020 from app.api.student_core import router as student_core_router
0021 from app.api.treehole import router as treehole_router
0022 from app.api.treehole import student_router as treehole_student_router
0023 from app.audit.writer import AuditWriter
0024 from app.config.settings import Settings
0025 from app.repositories.audit_repository import InMemoryAuditRepository
0026 from app.repositories.domain_data_repository import InMemoryDomainDataRepository
0027 from app.repositories.idempotency_repository import InMemoryIdempotencyRepository
0028 from app.repositories.session_repository import InMemorySessionRepository
0029 from app.schemas.envelope import ApiEnvelope
0030 from app.schemas.errors import ApiException
0031 from app.security.tokens import TokenManager
0032 from app.services.account_service import AccountService
0033 from app.services.admin_workbench_service import AdminWorkbenchService
0034 from app.services.ai_assist_service import AiAssistService
0035 from app.services.assessment_service import AssessmentService
0036 from app.services.auth_service import AuthService, WechatClient
0037 from app.services.bootstrap_service import BootstrapService
0038 from app.services.consent_service import ConsentService
0039 from app.services.idempotency_service import IdempotencyService
0040 from app.services.identity_access_service import IdentityAccessService
0041 from app.services.identity_service import IdentityService
0042 from app.services.mood_service import MoodService
0043 from app.services.quote_service import QuoteService
0044 from app.services.safety_service import SafetyService
0045 from app.services.support_resource_service import SupportResourceService
0046 from app.services.today_service import TodayService
0047 from app.services.treehole_service import TreeholeService

0050 class HealthData(BaseModel):
0051     service: str
0052     version: str
0053     environment_kind: str
0054     status: str

0057 def create_app(
0058     settings: Settings | None = None,
0059     *,
0060     wechat_client: WechatClient | None = None,
0061     session_repository: InMemorySessionRepository | None = None,
0062     token_manager: TokenManager | None = None,
0063     admin_workbench_service: AdminWorkbenchService | None = None,
0064     ai_assist_service: AiAssistService | None = None,
0065     domain_repository: InMemoryDomainDataRepository | None = None,
0066     idempotency_repository: InMemoryIdempotencyRepository | None = None,
0067     audit_repository: InMemoryAuditRepository | None = None,
0068 ) -> FastAPI:
0069     runtime_settings = settings or Settings.from_environment()
0070     runtime_sessions = session_repository or InMemorySessionRepository()
0071     runtime_tokens = token_manager or TokenManager()
```

backend/app/main.py

```
0072         runtime_settings.session_secret or "local-development-session-secret"
0073     )
0074     runtime_domain_repository = domain_repository or InMemoryDomainDataRepository()
0075     runtime_idempotency = IdempotencyService(idempotency_repository)
0076     runtime_audit = AuditWriter(
0077         audit_repository,
0078         environment_id=runtime_settings.cloudbase_env_id or "unconfigured",
0079     )
0080     app = FastAPI(title="心语 V2 API", version="0.1.0")
0081     app.state.settings = runtime_settings
0082     app.state.auth_service = AuthService(
0083         runtime_settings,
0084         session_repository=runtime_sessions,
0085         token_manager=runtime_tokens,
0086         wechat_client=wechat_client,
0087         domain_repository=runtime_domain_repository,
0088     )
0089     app.state.admin_workbench_service = admin_workbench_service or AdminWorkbenchService(
0090         runtime_settings
0091     )
0092     app.state.ai_assist_service = ai_assist_service or AiAssistService(runtime_settings)
0093     app.state.assessment_service = AssessmentService(
0094         settings=runtime_settings,
0095         repository=runtime_domain_repository,
0096         session_repository=runtime_sessions,
0097         token_manager=runtime_tokens,
0098         idempotency_service=runtime_idempotency,
0099         audit_writer=runtime_audit,
0100     )
0101     app.state.safety_service = SafetyService(
0102         settings=runtime_settings,
0103         repository=runtime_domain_repository,
0104         session_repository=runtime_sessions,
0105         token_manager=runtime_tokens,
0106         idempotency_service=runtime_idempotency,
0107         audit_writer=runtime_audit,
0108     )
0109     app.state.consent_service = ConsentService(
0110         settings=runtime_settings,
0111         repository=runtime_domain_repository,
0112         session_repository=runtime_sessions,
0113         token_manager=runtime_tokens,
0114         idempotency_service=runtime_idempotency,
0115         audit_writer=runtime_audit,
0116     )
0117     app.state.identity_service = IdentityService(
0118         settings=runtime_settings,
0119         repository=runtime_domain_repository,
0120         session_repository=runtime_sessions,
0121         token_manager=runtime_tokens,
0122         idempotency_service=runtime_idempotency,
```

backend/app/main.py

```
0123         audit_writer=runtime_audit,
0124     )
0125     support_service = SupportResourceService(
0126         settings=runtime_settings, repository=runtime_domain_repository
0127     )
0128     app.state.support_resource_service = support_service
0129     app.state.mood_service = MoodService(
0130         repository=runtime_domain_repository,
0131         session_repository=runtime_sessions,
0132         token_manager=runtime_tokens,
0133         idempotency_service=runtime_idempotency,
0134         audit_writer=runtime_audit,
0135     )
0136     app.state.today_service = TodayService(
0137         settings=runtime_settings,
0138         repository=runtime_domain_repository,
0139         session_repository=runtime_sessions,
0140         token_manager=runtime_tokens,
0141         quote_service=QuoteService(repository=runtime_domain_repository),
0142         support_resource_service=support_service,
0143     )
0144     app.state.account_service = AccountService(
0145         repository=runtime_domain_repository,
0146         session_repository=runtime_sessions,
0147         token_manager=runtime_tokens,
0148         idempotency_service=runtime_idempotency,
0149         audit_writer=runtime_audit,
0150     )
0151     app.state.bootstrap_service = BootstrapService(
0152         settings=runtime_settings,
0153         repository=runtime_domain_repository,
0154         session_repository=runtime_sessions,
0155         token_manager=runtime_tokens,
0156         identity_service=app.state.identity_service,
0157         today_service=app.state.today_service,
0158     )
0159     app.state.treehole_service = TreeholeService(
0160         settings=runtime_settings,
0161         repository=runtime_domain_repository,
0162         session_repository=runtime_sessions,
0163         token_manager=runtime_tokens,
0164         idempotency_service=runtime_idempotency,
0165         audit_writer=runtime_audit,
0166         consent_service=app.state.consent_service,
0167     )
0168     app.state.identity_access_service = IdentityAccessService(
0169         repository=runtime_domain_repository,
0170         session_repository=runtime_sessions,
0171         token_manager=runtime_tokens,
0172         idempotency_service=runtime_idempotency,
0173         audit_writer=runtime_audit,
```

backend/app/main.py

```
0174         identity_service=app.state.identity_service,
0175     )
0177     register_exception_handlers(app)
0179     @app.middleware("http")
0180     async def request_id_middleware(request: Request, call_next: Any) -> Any:
0181         try:
0182             request.state.request_id = resolve_request_id(request.headers.get("X-Request-Id"))
0183         except ApiException as error:
0184             request.state.request_id = f"req_invalid_{resolve_request_id(None)[4:]}"
0185             response = _error_response(request, error)
0186             response.headers["X-Request-Id"] = request.state.request_id
0187             return response
0188         response = await call_next(request)
0189         response.headers["X-Request-Id"] = request.state.request_id
0190         return response
0192     @app.get("/api/v1/health")
0193     async def health(request: Request) -> ApiEnvelope[HealthData]:
0194         settings = request.app.state.settings
0195         data = HealthData(
0196             service="xinyu-v2-backend",
0197             version="0.1.0",
0198             environment_kind=settings.environment_kind.value,
0199             status="ok" if settings.configuration_status == "ready" else "degraded",
0200         )
0201         return ApiEnvelope.success(request_id=request.state.request_id, data=data)
0203     app.include_router(auth_router)
0204     app.include_router(me_router)
0205     app.include_router(admin_auth_router)
0206     app.include_router(admin_workbench_router)
0207     app.include_router(admin_identity_router)
0208     app.include_router(assessment_router)
0209     app.include_router(assessment_contract_router)
0210     app.include_router(treehole_router)
0211     app.include_router(treehole_student_router)
0212     app.include_router(student_core_router)
0213     return app
0216 app = create_app()
```

miniprogram/pages/onboarding/index.ts

```
0001 import { loginWithWechat, recordBasicConsent } from '../../services/auth';
0003 Page({
0004     data: { loading: false, error: '', agreed: false },
0005     toggleAgree() {
0006         this.setData({ agreed: !this.data.agreed, error: '' });
0007     },
0008     async begin() {
0009         if (!this.data.agreed) {
0010             this.setData({ error: '请先阅读并同意基础服务说明' });
0011             return;
0012         }
0013         this.setData({ loading: true, error: '' });
0014         try {
0015             const session = await loginWithWechat();
```

miniprogram/pages/onboarding/index.ts

```
0016     if (!session.basicConsent) await recordBasicConsent();
0017     if (session.identityVerified) wx.switchTab({ url: '/pages/today/index' });
0018     else
0019         wx.redirectTo({ url: '/pages/identity-verification/index?from=onboarding' });
0020   } catch (error) {
0021     this.setData({
0022       error: error instanceof Error ? error.message : '暂时无法进入，请稍后重试',
0023     });
0024   } finally {
0025     this.setData({ loading: false });
0026   }
0027 },
0028 leave() {
0029   wx.navigateBack({ fail: () => wx.switchTab({ url: '/pages/today/index' }) });
0030 },
0031 });
```

miniprogram/pages/identity-verification/index.ts

```
0001 import { verifyIdentity } from '../../services/auth';
0003 Page({
0004   data: { name: '', studentNumber: '', loading: false, error: '', from: '' },
0005   onLoad(query: Record<string, string>) {
0006     this.setData({ from: query.from ?? '' });
0007   },
0008   onNameInput(event: WechatMiniprogram.Input) {
0009     this.setData({ name: event.detail.value, error: '' });
0010   },
0011   onStudentInput(event: WechatMiniprogram.Input) {
0012     this.setData({ studentNumber: event.detail.value, error: '' });
0013   },
0014   async submit() {
0015     if (!this.data.name.trim() || !this.data.studentNumber.trim()) {
0016       this.setData({ error: '请填写姓名和学号' });
0017       return;
0018     }
0019     this.setData({ loading: true, error: '' });
0020     try {
0021       await verifyIdentity(this.data.name.trim(), this.data.studentNumber.trim());
0022       wx.switchTab({ url: '/pages/today/index' });
0023     } catch (error) {
0024       this.setData({
0025         error: error instanceof Error ? error.message : '身份核验服务暂时不可用',
0026       });
0027     } finally {
0028       this.setData({ loading: false });
0029     }
0030   },
0031   cancel() {
0032     wx.navigateBack({ fail: () => wx.switchTab({ url: '/pages/today/index' }) });
0033   },
0034 });
```

miniprogram/pages/today/index.ts

```
0001 import { fetchToday, saveMood } from '../../services/today';
0002 import { MOOD_OPTIONS, moodDateKey } from '../../services/mood';
```

miniprogram/pages/today/index.ts

```
0003 import { sessionStore } from '../../stores/session';
0004 import type { MoodRecord, QuoteProjection, TodayObservation } from '../../types/api';
0006 Page({
0007   data: {
0008     loading: true,
0009     loaded: false,
0010     recordDate: '',
0011     saving: false,
0012     error: '',
0013     moodError: '',
0014     quote: null as QuoteProjection | null,
0015     mood: null as MoodRecord | null,
0016     observations: [] as TodayObservation[],
0017     selectedMood: '',
0018     showMoodSheet: false,
0019     todayDate: '',
0020     greeting: '',
0021     moods: [...MOOD_OPTIONS],
0022   },
0023   onShow() {
0024     this.setDateContext();
0025     if (this.data.showMoodSheet) wx.hideTabBar({ animation: false });
0026     if (!this.data.saving) this.load();
0027   },
0028   onHide() {
0029     wx.showTabBar({ animation: false });
0030   },
0031   onUnload() {
0032     wx.showTabBar({ animation: false });
0033   },
0034   setDateContext() {
0035     const now = new Date();
0036     const weekdays = ['周日', '周一', '周二', '周三', '周四', '周五', '周六'];
0037     const hour = now.getHours();
0038     this.setData({
0039       todayDate: `${now.getMonth() + 1}月${now.getDate()}日 ${weekdays[now.getDay()]}`,
0040       greeting:
0041         hour < 6
0042           ? '夜深了，先照顾好自己。'
0043           : hour < 12
0044             ? '早上好，按自己的节奏开始今天。'
0045             : hour < 18
0046               ? '下午好，留一点时间给自己。'
0047               : '晚上好，今天也辛苦了。',
0048     });
0049   },
0050   async load() {
0051     const recordDate = moodDateKey();
0052     if (this.data.recordDate !== recordDate)
0053       this.setData({
0054         loaded: false,
```

miniprogram/pages/today/index.ts

```
0055         mood: null,
0056         observations: [],
0057         selectedMood: '',
0058         moodError: '',
0059         recordDate,
0060     });
0061     this.setData({ loading: true, error: '' });
0062     try {
0063         const today = await fetchToday();
0064         this.setData({
0065             quote: today.quote,
0066             mood: today.mood,
0067             observations: today.observations,
0068             loaded: true,
0069         });
0070     } catch {
0071         this.setData({ error: '暂时无法读取今天的内容，请重新试试。' });
0072     } finally {
0073         this.setData({ loading: false });
0074     }
0075 },
0076 selectMood(event: WechatMiniprogram.BaseEvent) {
0077     if (this.data.mood || this.data.saving) return;
0078     const code = (event.currentTarget as WechatMiniprogram.Target).dataset
0079         .mood as string;
0080     if (MOOD_OPTIONS.some((item) => item.code === code))
0081         this.setData({ selectedMood: code, moodError: '' });
0082 },
0083 async saveMood() {
0084     if (this.data.saving || this.data.mood) return;
0085     if (!this.data.selectedMood) {
0086         this.setData({ moodError: '请先选择今天的心情' });
0087         return;
0088     }
0089     if (sessionStore.get()?.accountStatus === 'recovery') {
0090         this.setData({ moodError: '账户处于恢复期，恢复后才能记录' });
0091         return;
0092     }
0093     this.setData({ saving: true, moodError: '' });
0094     try {
0095         const mood = await saveMood(this.data.selectedMood);
0096         this.setData({ mood, selectedMood: '', moodError: '', loaded: true });
0097     } catch {
0098         this.setData({ moodError: '暂时没有记下这次选择' });
0099     } finally {
0100         this.setData({ saving: false });
0101     }
0102 },
0103 openObservation(event: WechatMiniprogram.BaseEvent) {
0104     const key = (event.currentTarget as WechatMiniprogram.Target).dataset.key;
0105     const observation = this.data.observations.find((item) => item.key === key);
```

miniprogram/pages/today/index.ts

```
0106     if (observation?.safetyEntryRequired) {
0107         wx.navigateTo({ url: '/pages/support-resources/index?context=safety' });
0108         return;
0109     }
0110     this.goAssessment();
0111 },
0112 preventScroll() {},
0113 goAssessment() {
0114     wx.switchTab({ url: '/pages/assessment-center/index' });
0115 },
0116 openMoodSheet() {
0117     if (!this.data.loaded || this.data.loading) return;
0118     wx.hideTabBar({ animation: false });
0119     this.setData({ showMoodSheet: true, moodError: '' });
0120 },
0121 closeMoodSheet() {
0122     if (!this.data.saving) {
0123         wx.showTabBar({ animation: false });
0124         this.setData({ showMoodSheet: false, selectedMood: '', moodError: '' });
0125     }
0126 },
0127 goTreehole() {
0128     wx.switchTab({ url: '/pages/treehole/index' });
0129 },
0130 goSupport() {
0131     wx.navigateTo({ url: '/pages/support-resources/index' });
0132 },
0133 goMy() {
0134     wx.switchTab({ url: '/pages/my/index' });
0135 },
0136 });
```

miniprogram/pages/assessment-center/index.ts

```
0001 import { fetchModules, startAssessment } from '../../services/assessment';
0002 import type { AssessmentModule } from '../../types/api';
0003 import { sessionStore } from '../../stores/session';
0004
0005 Page({
0006     data: { loading: true, error: '', modules: [] as AssessmentModule[] },
0007     onShow() {
0008         this.load();
0009     },
0010     async load() {
0011         this.setData({ loading: true, error: '' });
0012         try {
0013             this.setData({ modules: await fetchModules() });
0014         } catch (error) {
0015             this.setData({
0016                 error: error instanceof Error ? error.message : '自测目录暂时不可用',
0017             });
0018         } finally {
0019             this.setData({ loading: false });
0020         }
0021     },
```

miniprogram/pages/assessment-center/index.ts

```
0022   async begin(event: WechatMiniprogram.BaseEvent) {
0023     const key = (event.currentTarget as WechatMiniprogram.Target).dataset
0024       .key as AssessmentModule['key'];
0025     const current = sessionStore.get();
0026     if (!current?.identityVerified) {
0027       wx.navigateTo({
0028         url: `/pages/identity-verification/index?from=assessment-${key}`,
0029       });
0030       return;
0031     }
0032     if (current.accountStatus === 'recovery') {
0033       this.setData({ error: '账户处于恢复期，恢复后才能开始新的观察' });
0034       return;
0035     }
0036     try {
0037       const session = await startAssessment(key);
0038       wx.navigateTo({
0039         url: `/pages/assessment-answer/index?sessionId=${session.id}&module=${session.module}
&title=${encodeURIComponent(session.title)}&questions=${
encodeURIComponent(JSON.stringify(session.questions))}`,
0040       });
0041     } catch (error) {
0042       this.setData({
0043         error: error instanceof Error ? error.message : '暂时无法开始本次观察',
0044       });
0045     }
0046   },
0047   history() {
0048     wx.navigateTo({ url: '/pages/history/index' });
0049   },
0050 });
```

miniprogram/pages/assessment-answer/index.ts

```
0001 import { submitAssessment } from '../../services/assessment';
0002 import { assessmentStore } from '../../stores/assessment';
0003 import type { AssessmentQuestion, AssessmentModule } from '../../types/api';
0004
0005 Page({
0006   data: {
0007     title: '答题',
0008     sessionId: '',
0009     module: 'phq9' as AssessmentModule['key'],
0010     questions: [] as AssessmentQuestion[],
0011     currentIndex: 0,
0012     selected: -1,
0013     submitting: false,
0014     error: '',
0015     showLeave: false,
0016   },
0017   onLoad(query: Record<string, string>) {
0018     const questions = JSON.parse(
0019       decodeURIComponent(query.questions ?? '[]'),
0020     ) as AssessmentQuestion[];
0021     assessmentStore.start({
0022       sessionId: query.sessionId ?? '',
0023       module: (query.module ?? 'phq9') as AssessmentModule['key'],
```

miniprogram/pages/assessment-answer/index.ts

```
0024     title: decodeURIComponent(query.title ?? '答题'),
0025     questions,
0026   });
0027   this.sync();
0028 },
0029 onShow() {
0030   this.sync();
0031 },
0032 sync() {
0033   const state = assessmentStore.get();
0034   if (state)
0035     this.setData({
0036       title: state.title,
0037       sessionId: state.sessionId,
0038       module: state.module,
0039       questions: state.questions,
0040       currentIndex: state.currentIndex,
0041       selected: state.answers[state.currentIndex] ?? -1,
0042     });
0043 },
0044 choose(event: WechatMiniprogram.BaseEvent) {
0045   const value = Number(
0046     (event.currentTarget as WechatMiniprogram.Target).dataset.value,
0047   );
0048   const state = assessmentStore.get();
0049   if (!state) return;
0050   assessmentStore.setAnswer(state.currentIndex, value);
0051   this.setData({ selected: value });
0052   if (state.module === 'phq9' && state.currentIndex === 8 && value > 0) {
0053     wx.navigateTo({
0054       url: `/pages/safety-confirmation/index?sessionId=${state.sessionId}`,
0055     });
0056     return;
0057   }
0058   if (state.currentIndex < state.questions.length - 1) {
0059     state.currentIndex += 1;
0060     this.setData({
0061       currentIndex: state.currentIndex,
0062       selected: state.answers[state.currentIndex] ?? -1,
0063     });
0064   }
0065 },
0066 previous() {
0067   const state = assessmentStore.get();
0068   if (!state || state.currentIndex === 0) return;
0069   state.currentIndex -= 1;
0070   this.setData({
0071     currentIndex: state.currentIndex,
0072     selected: state.answers[state.currentIndex] ?? -1,
0073   });
0074 },
```

miniprogram/pages/assessment-answer/index.ts

```
0075     next() {
0076         const state = assessmentStore.get();
0077         if (!state || state.answers[state.currentIndex] === undefined) {
0078             this.setData({ error: '请先选择当前答案' });
0079             return;
0080         }
0081         if (state.currentIndex < state.questions.length - 1) {
0082             state.currentIndex += 1;
0083             this.setData({
0084                 currentIndex: state.currentIndex,
0085                 selected: state.answers[state.currentIndex] ?? -1,
0086                 error: '',
0087             });
0088         }
0089     },
0090     async submit() {
0091         const state = assessmentStore.get();
0092         if (!state) return;
0093         if (
0094             state.answers.length < state.questions.length ||
0095             state.answers.some((answer) => answer === undefined)
0096         ) {
0097             this.setData({ error: '还有题目未完成，请回到第一道未答题' });
0098             return;
0099         }
0100         this.setData({ submitting: true, error: '' });
0101         try {
0102             const result = await submitAssessment(
0103                 state.sessionId,
0104                 state.module,
0105                 state.answers,
0106                 state.safetyState,
0107             );
0108             assessmentStore.setResult(result);
0109             wx.redirectTo({
0110                 url: `/pages/assessment-result/index?result=${encodeURIComponent(JSON.stringify(result))}`
0111             });
0112         } catch (error) {
0113             this.setData({
0114                 error:
0115                     error instanceof Error ? error.message : '结果暂时没有保存成功，请稍后重试',
0116             });
0117         } finally {
0118             this.setData({ submitting: false });
0119         }
0120     },
0121     askLeave() {
0122         this.setData({ showLeave: true });
0123     },
0124     cancelLeave() {
0125         this.setData({ showLeave: false });
```

miniprogram/pages/assessment-answer/index.ts

```
0126     },
0127     leave() {
0128         assessmentStore.clear();
0129         this.setData({ showLeave: false });
0130         wx.switchTab({ url: '/pages/assessment-center/index' });
0131     },
0132 });
```

miniprogram/pages/assessment-result/index.ts

```
0001 import type { AssessmentResult } from '../../types/api';
0003 Page({
0004     data: { result: null as AssessmentResult | null, error: '' },
0005     onLoad(query: Record<string, string>) {
0006         try {
0007             this.setData({
0008                 result: JSON.parse(
0009                     decodeURIComponent(query.result ?? ''),
0010                 ) as AssessmentResult,
0011             });
0012         } catch {
0013             this.setData({ error: '本次记录暂时无法展示，请稍后重试' });
0014         }
0015     },
0016     support() {
0017         wx.navigateTo({ url: '/pages/support-resources/index' });
0018     },
0019     history() {
0020         wx.navigateTo({ url: '/pages/history/index' });
0021     },
0022     again() {
0023         wx.switchTab({ url: '/pages/assessment-center/index' });
0024     },
0025 });
```

miniprogram/pages/safety-confirmation/index.ts

```
0001 import { confirmSafety } from '../../services/assessment';
0002 import { assessmentStore } from '../../stores/assessment';
0004 Page({
0005     data: { loading: false, error: '', sessionId: '', selected: '' },
0006     onLoad(query: Record<string, string>) {
0007         this.setData({
0008             sessionId: query.sessionId ?? assessmentStore.get()?.sessionId ?? '',
0009         });
0010     },
0011     async choose(event: WechatMiniprogram.BaseEvent) {
0012         const state = (event.currentTarget as WechatMiniprogram.Target).dataset.state as
0013             'can_be_safe' | 'uncertain' | 'cannot_be_safe';
0014         this.setData({ loading: true, error: '', selected: state });
0015         try {
0016             const current = assessmentStore.get();
0017             await confirmSafety(
0018                 this.data.sessionId,
0019                 state,
0020                 current?.answers.slice(0, 9) ?? [],
0021             );
0022         }
```

miniprogram/pages/safety-confirmation/index.ts

```
0022     assessmentStore.setSafety(state);
0023     if (state === 'can_be_safe') wx.navigateBack();
0024     else
0025         wx.redirectTo({
0026             url: `/pages/support-resources/index?context=safety&state=${state}&sessionId=${
0027                 {this.data.sessionId}`,
0028             });
0029     } catch (error) {
0030         this.setData({
0031             error:
0032                 error instanceof Error ? error.message : '安全确认没有保存成功，请重试',
0033         });
0034     } finally {
0035         this.setData({ loading: false });
0036     }
0037 });
```

miniprogram/pages/support-resources/index.ts

```
0001 import {
0002     acknowledgeSupportResources,
0003     fetchResources,
0004 } from '../../services/assessment';
0005 import type { SupportResource } from '../../types/api';
0006
0007 Page({
0008     data: {
0009         loading: true,
0010         error: '',
0011         context: 'ordinary' as 'ordinary' | 'safety',
0012         safetyState: '' as string,
0013         sessionId: '',
0014         resources: [] as SupportResource[],
0015     },
0016     onLoad(query: Record<string, string>) {
0017         this.setData({
0018             context: query.context === 'safety' ? 'safety' : 'ordinary',
0019             safetyState: query.state ?? '',
0020             sessionId: query.sessionId ?? '',
0021         });
0022     },
0023     onShow() {
0024         this.load();
0025     },
0026     async load() {
0027         this.setData({ loading: true, error: '' });
0028         try {
0029             this.setData({ resources: await fetchResources(this.data.context) });
0030             if (
0031                 this.data.context === 'safety' &&
0032                 this.data.safetyState === 'uncertain' &&
0033                 this.data.sessionId
0034             )
0035                 await acknowledgeSupportResources(this.data.sessionId);
0036         } catch (error) {
```

miniprogram/pages/support-resources/index.ts

```
0037     this.setData({
0038         error: error instanceof Error ? error.message : '支持资源暂时不可用',
0039     });
0040     } finally {
0041         this.setData({ loading: false });
0042     }
0043 },
0044 call(event: WechatMiniprogram.BaseEvent) {
0045     const phone = (event.currentTarget as WechatMiniprogram.Target).dataset
0046         .phone as string;
0047     if (phone) wx.makePhoneCall({ phoneNumber: phone });
0048 },
0049 backToAnswer() {
0050     wx.navigateBack({ delta: 1 });
0051 },
0052 });
```

miniprogram/pages/history/index.ts

```
0001 import { fetchHistory } from '../../services/me';
0002 import type { HistoryItem } from '../../types/api';
0004 Page({
0005     data: { loading: true, error: '', items: [] as HistoryItem[] },
0006     onShow() {
0007         this.load();
0008     },
0009     async load() {
0010         try {
0011             this.setData({ loading: true, error: '' });
0012             this.setData({ items: await fetchHistory() });
0013         } catch (error) {
0014             this.setData({
0015                 error: error instanceof Error ? error.message : '历史记录暂时不可用',
0016             });
0017         } finally {
0018             this.setData({ loading: false });
0019         }
0020     },
0021     start() {
0022         wx.switchTab({ url: '/pages/assessment-center/index' });
0023     },
0024     open(event: WechatMiniprogram.BaseEvent) {
0025         const target = event.currentTarget as WechatMiniprogram.Target;
0026         const id = target.dataset.id as string;
0027         const category = target.dataset.category as string;
0028         wx.navigateTo({
0029             url: `/pages/delete-confirmation/index?id=${id}&category=${category}`,
0030         });
0031     },
0032 });
```

miniprogram/pages/treehole/index.ts

```
0001 import { fetchPosts } from '../../services/treehole';
0002 import { displayTime } from '../../services/display-time';
0003 import type { TreeholePost } from '../../types/api';
0004 import { sessionStore } from '../../stores/session';
```

miniprogram/pages/treehole/index.ts

```
0006 Page({
0007   data: {
0008     loading: true,
0009     error: '',
0010     posts: [] as Array<TreeholePost & { displayTime: string }>,
0011   },
0012   onShow() {
0013     this.load();
0014   },
0015   async load() {
0016     this.setData({ loading: true, error: '' });
0017     try {
0018       const posts = await fetchPosts();
0019       this.setData({
0020         posts: posts.map((post) => ({
0021           ...post,
0022           displayTime: displayTime(post.createdAt),
0023         })),
0024       });
0025     } catch {
0026       this.setData({ error: '暂时没有加载到内容' });
0027     } finally {
0028       this.setData({ loading: false });
0029     }
0030   },
0031   open(event: WechatMiniprogram.BaseEvent) {
0032     const id = (event.currentTarget as WechatMiniprogram.Target).dataset.id as string;
0033     wx.navigateTo({ url: `/pages/treehole-detail/index?id=${id}` });
0034   },
0035   publish() {
0036     const user = sessionStore.getUser();
0037     if (!user.identityVerified) {
0038       wx.navigateTo({ url: '/pages/identity-verification/index?from=treehole' });
0039       return;
0040     }
0041     if (!user.communityConsent) {
0042       wx.navigateTo({ url: '/pages/privacy/index?focus=community' });
0043       return;
0044     }
0045     wx.navigateTo({ url: '/pages/treehole-publish/index' });
0046   },
0047 });
```

miniprogram/pages/treehole-publish/index.ts

```
0001 import { publishPost } from '../../services/treehole';
0003 Page({
0004   data: { body: '', loading: false, error: '', checked: false },
0005   onInput(event: WechatMiniprogram.TextareaInput) {
0006     this.setData({ body: event.detail.value, error: '', checked: false });
0007   },
0008   check() {
0009     if (!this.data.body.trim()) {
0010       this.setData({ error: '请先写下一段内容' });
0011     }
0012   }
0013 });
```

miniprogram/pages/treehole-publish/index.ts

```
0011     return;
0012   }
0013   if (this.data.body.length > 1000) {
0014     this.setData({ error: '内容最多 1000 字，请适当精简' });
0015     return;
0016   }
0017   this.setData({ checked: true, error: '' });
0018 },
0019 async submit() {
0020   if (!this.data.checked) {
0021     this.check();
0022     if (!this.data.checked) return;
0023   }
0024   this.setData({ loading: true, error: '' });
0025   try {
0026     const post = await publishPost(this.data.body.trim());
0027     wx.redirectTo({
0028       url: `/pages/treehole-status/index?id=${post.id}&status=${post.status}`,
0029     });
0030   } catch (error) {
0031     this.setData({
0032       error:
0033         error instanceof Error ? error.message : '内容没有提交成功，请稍后重试',
0034     });
0035   } finally {
0036     this.setData({ loading: false });
0037   }
0038 },
0039 });
```

miniprogram/pages/treehole-status/index.ts

```
0001 Page({
0002   data: { status: '', id: '', message: '' },
0003   onLoad(query: Record<string, string>) {
0004     const status = query.status ?? '';
0005     const messages: Record<string, string> = {
0006       checking: '内容正在检查，检查完成前不会出现在公开列表。',
0007       published: '内容已公开展示。',
0008       protected: '内容已保护展示，互动范围可能受限。',
0009       pending_confirmation:
0010         '自动检查尚未完成，这条内容会等待人工处理，暂时不会公开。',
0011       unpublished: '这条内容暂未公开，内部理由不会在这里展示。',
0012       safety_priority: '这条内容优先进入支持处理，完整正文暂不展示。',
0013       deleted: '这条内容已删除，正文不会恢复。',
0014     };
0015     this.setData({
0016       status,
0017       id: query.id ?? '',
0018       message: messages[status] ?? '当前状态暂时不可用',
0019     });
0020   },
0021   back() {
0022     wx.navigateBack({ fail: () => wx.switchTab({ url: '/pages/treehole/index' }) });
```

miniprogram/pages/treehole-status/index.ts

```
0023     },
0024   });
```

miniprogram/pages/treehole-detail/index.ts

```
0001 import { fetchPost, respondToPost } from '../../services/treehole';
0002 import type { TreeholePost } from '../../types/api';
0003 import { sessionStore } from '../../stores/session';

0005 Page({
0006   data: {
0007     loading: true,
0008     error: '',
0009     post: null as TreeholePost | null,
0010     id: '',
0011     response: '',
0012     responding: false,
0013   },
0014   onLoad(query: Record<string, string>) {
0015     this.setData({ id: query.id ?? '' });
0016     this.load();
0017   },
0018   async load() {
0019     try {
0020       this.setData({ loading: true, error: '' });
0021       this.setData({ post: await fetchPost(this.data.id) });
0022     } catch (error) {
0023       this.setData({
0024         error: error instanceof Error ? error.message : '帖子暂时不可见',
0025       });
0026     } finally {
0027       this.setData({ loading: false });
0028     }
0029   },
0030   input(event: WechatMiniprogram.TextareaInput) {
0031     this.setData({ response: event.detail.value });
0032   },
0033   async submitResponse() {
0034     if (!this.data.response.trim() || !this.data.post) return;
0035     const user = sessionStore.getUser();
0036     if (!user.communityConsent) {
0037       this.setData({ error: '社区同意已撤回，暂时不能回应' });
0038       return;
0039     }
0040     this.setData({ responding: true, error: '' });
0041     try {
0042       await respondToPost(this.data.post.id, this.data.response.trim());
0043       this.setData({ response: '' });
0044       await this.load();
0045     } catch (error) {
0046       this.setData({
0047         error:
0048           error instanceof Error ? error.message : '回应没有提交成功，请稍后重试',
0049       });
0050     } finally {
```

miniprogram/pages/treehole-detail/index.ts

```
0051     this.setData({ responding: false });
0052   }
0053 },
0054 manage() {
0055   wx.navigateTo({
0056     url: `/pages/treehole-status/index?id=${this.data.id}&status=${this.data.post?.status ??
    ''}&mine=true`,
0057   });
0058 },
0059 });
```

miniprogram/pages/my/index.ts

```
0001 import { fetchProfile } from '../../services/me';
0002 import type { UserProjection } from '../../types/api';
0004 Page({
0005   data: { loading: true, error: '', profile: null as UserProjection | null },
0006   onShow() {
0007     this.load();
0008   },
0009   async load() {
0010     try {
0011       this.setData({ loading: true, error: '' });
0012       this.setData({ profile: await fetchProfile() });
0013     } catch (error) {
0014       this.setData({
0015         error: error instanceof Error ? error.message : '账户信息暂时不可用',
0016       });
0017     } finally {
0018       this.setData({ loading: false });
0019     }
0020   },
0021   open(event: WechatMiniprogram.BaseEvent) {
0022     const path = (event.currentTarget as WechatMiniprogram.Target).dataset
0023       .path as string;
0024     wx.navigateTo({ url: path });
0025   },
0026 });
```

miniprogram/pages/anonymous-identity/index.ts

```
0001 import { fetchProfile } from '../../services/me';
0002 Page({
0003   data: { loading: true, error: '', displayName: '' },
0004   onShow() {
0005     this.load();
0006   },
0007   async load() {
0008     try {
0009       const profile = await fetchProfile();
0010       this.setData({ displayName: profile.displayName ?? '匿名同学' });
0011     } catch (error) {
0012       this.setData({
0013         error: error instanceof Error ? error.message : '匿名身份暂时不可用',
0014       });
0015     } finally {
0016       this.setData({ loading: false });
0017     }
0018   }
0019 });
```

miniprogram/pages/anonymous-identity/index.ts

```
0018     },
0019   });
```

miniprogram/pages/my-content/index.ts

```
0001 import { fetchMyPosts } from '../../services/treehole';
0002 import type { TreeholePost } from '../../types/api';
0003 Page({
0004   data: { loading: true, error: '', posts: [] as TreeholePost[] },
0005   onShow() {
0006     this.load();
0007   },
0008   async load() {
0009     try {
0010       this.setData({ posts: await fetchMyPosts() });
0011     } catch (error) {
0012       this.setData({
0013         error: error instanceof Error ? error.message : '我的内容暂时不可用',
0014       });
0015     } finally {
0016       this.setData({ loading: false });
0017     }
0018   },
0019   open(event: WechatMiniprogram.BaseEvent) {
0020     const id = (event.currentTarget as WechatMiniprogram.Target).dataset.id as string;
0021     wx.navigateTo({ url: `/pages/treehole-detail/index?id=${id}` });
0022   },
0023   publish() {
0024     wx.navigateTo({ url: '/pages/treehole-publish/index' });
0025   },
0026 });
```

miniprogram/pages/privacy/index.ts

```
0001 import { fetchProfile } from '../../services/me';
0002 import { updateCommunityConsent } from '../../services/auth';
0003 Page({
0004   data: { loading: true, saving: false, error: '', communityConsent: false },
0005   onShow() {
0006     this.load();
0007   },
0008   async load() {
0009     try {
0010       const profile = await fetchProfile();
0011       this.setData({ communityConsent: profile.communityConsent });
0012     } catch (error) {
0013       this.setData({
0014         error: error instanceof Error ? error.message : '隐私状态暂时不可用',
0015       });
0016     } finally {
0017       this.setData({ loading: false });
0018     }
0019   },
0020   async toggleCommunity() {
0021     this.setData({ saving: true, error: '' });
0022     try {
0023       const profile = await updateCommunityConsent(!this.data.communityConsent);
```

miniprogram/pages/privacy/index.ts

```
0024     this.setData({ communityConsent: profile.communityConsent });
0025   } catch (error) {
0026     this.setData({
0027       error: error instanceof Error ? error.message : '同意状态没有更新成功',
0028     });
0029   } finally {
0030     this.setData({ saving: false });
0031   }
0032 },
0033 stop() {
0034   wx.navigateTo({ url: '/pages/account-stop/index' });
0035 },
0036 });
```

miniprogram/pages/account-stop/index.ts

```
0001 import { stopAccount } from '../../services/auth';
0002 Page({
0003   data: { confirmText: '', loading: false, error: '' },
0004   input(event: WechatMiniprogram.Input) {
0005     this.setData({ confirmText: event.detail.value, error: '' });
0006   },
0007   async submit() {
0008     if (this.data.confirmText !== '停止使用') {
0009       this.setData({ error: '请输入“停止使用”确认此操作' });
0010       return;
0011     }
0012     this.setData({ loading: true, error: '' });
0013     try {
0014       await stopAccount();
0015       wx.redirectTo({ url: '/pages/account-recover/index' });
0016     } catch (error) {
0017       this.setData({
0018         error: error instanceof Error ? error.message : '账户暂时没有停止成功',
0019       });
0020     } finally {
0021       this.setData({ loading: false });
0022     }
0023   },
0024 });
```

miniprogram/pages/account-recover/index.ts

```
0001 import { recoverAccount } from '../../services/auth';
0002 Page({
0003   data: { loading: false, error: '' },
0004   async submit() {
0005     this.setData({ loading: true, error: '' });
0006     try {
0007       await recoverAccount();
0008       wx.switchTab({ url: '/pages/today/index' });
0009     } catch (error) {
0010       this.setData({
0011         error: error instanceof Error ? error.message : '账户暂时无法恢复',
0012       });
0013     } finally {
0014       this.setData({ loading: false });
0015     }
0016   }
0017 });
```

miniprogram/pages/account-recover/index.ts

```
0015     }  
0016   },  
0017 });
```

miniprogram/pages/delete-confirmation/index.ts

```
0001 import { deleteHistoryItem } from '../../services/me';  
0002 import type { HistoryItem } from '../../types/api';  
0003 Page({  
0004   data: {  
0005     id: '',  
0006     category: 'assessment' as HistoryItem['category'],  
0007     loading: false,  
0008     error: '',  
0009   },  
0010   onLoad(query: Record<string, string>) {  
0011     this.setData({  
0012       id: query.id ?? '',  
0013       category: (query.category ?? 'assessment') as HistoryItem['category'],  
0014     });  
0015   },  
0016   async remove() {  
0017     if (!this.data.id) return;  
0018     this.setData({ loading: true, error: '' });  
0019     try {  
0020       await deleteHistoryItem(this.data.id, this.data.category);  
0021       wx.navigateBack();  
0022     } catch (error) {  
0023       this.setData({  
0024         error: error instanceof Error ? error.message : '删除没有成功，请稍后重试',  
0025       });  
0026     } finally {  
0027       this.setData({ loading: false });  
0028     }  
0029   },  
0030   cancel() {  
0031     wx.navigateBack();  
0032   },  
0033 });
```

miniprogram/app.json

```
0001 {  
0002   "pages": [  
0003     "pages/onboarding/index",  
0004     "pages/identity-verification/index",  
0005     "pages/today/index",  
0006     "pages/assessment-center/index",  
0007     "pages/assessment-answer/index",  
0008     "pages/assessment-result/index",  
0009     "pages/safety-confirmation/index",  
0010     "pages/support-resources/index",  
0011     "pages/history/index",  
0012     "pages/treehole/index",  
0013     "pages/treehole-publish/index",  
0014     "pages/treehole-status/index",  
0015     "pages/treehole-detail/index",
```

miniprogram/app.json

```
0016     "pages/my/index",
0017     "pages/anonymous-identity/index",
0018     "pages/my-content/index",
0019     "pages/privacy/index",
0020     "pages/account-stop/index",
0021     "pages/account-recover/index",
0022     "pages/delete-confirmation/index"
0023   ],
0024   "window": {
0025     "navigationBarTitleText": "心语",
0026     "navigationBarBackgroundColor": "#F7F4EE",
0027     "navigationBarTextStyle": "black",
0028     "backgroundColor": "#F7F4EE",
0029     "backgroundTextStyle": "light"
0030   },
0031   "tabBar": {
0032     "color": "#5F6E6A",
0033     "selectedColor": "#2E6F68",
0034     "backgroundColor": "#FFFFFF",
0035     "borderStyle": "white",
0036     "list": [
0037       {
0038         "pagePath": "pages/today/index",
0039         "text": "今日",
0040         "iconPath": "assets/tabbar/today.png",
0041         "selectedIconPath": "assets/tabbar/today-selected.png"
0042       },
0043       {
0044         "pagePath": "pages/assessment-center/index",
0045         "text": "自测",
0046         "iconPath": "assets/tabbar/assessment.png",
0047         "selectedIconPath": "assets/tabbar/assessment-selected.png"
0048       },
0049       {
0050         "pagePath": "pages/treehole/index",
0051         "text": "树洞",
0052         "iconPath": "assets/tabbar/treehole.png",
0053         "selectedIconPath": "assets/tabbar/treehole-selected.png"
0054       },
0055       {
0056         "pagePath": "pages/my/index",
0057         "text": "我的",
0058         "iconPath": "assets/tabbar/my.png",
0059         "selectedIconPath": "assets/tabbar/my-selected.png"
0060       }
0061     ]
0062   },
0063   "sitemapLocation": "sitemap.json"
0064 }
```

miniprogram/app.wxss

```
0001 page {
0002   --xinyu-color-paper: #f7f4ee;
```

miniprogram/app.wxss

```
0003  --xinyu-color-surface: #ffffff;
0004  --xinyu-color-text: #1f2d2b;
0005  --xinyu-color-text-secondary: #5f6e6a;
0006  --xinyu-color-text-muted: #8c9a96;
0007  --xinyu-color-primary: #2e6f68;
0008  --xinyu-color-primary-pressed: #245b55;
0009  --xinyu-color-primary-soft: #e8f0ec;
0010  --xinyu-color-divider: #d7e4df;
0011  --xinyu-color-safety: #b85c4a;
0012  --xinyu-color-safety-soft: #f5e7e3;
0013  --xinyu-color-caution: #c58a3d;
0014  --xinyu-color-caution-soft: #f6eedb;
0015  --xinyu-font-sans:
0016    -apple-system, BlinkMacSystemFont, 'PingFang SC', 'Hiragino Sans GB',
0017    'Microsoft YaHei', sans-serif;
0018  --xinyu-space-1: 4px;
0019  --xinyu-space-2: 8px;
0020  --xinyu-space-3: 12px;
0021  --xinyu-space-4: 16px;
0022  --xinyu-space-5: 20px;
0023  --xinyu-space-6: 24px;
0024  --xinyu-space-8: 32px;
0025  --xinyu-space-10: 40px;
0026  --xinyu-space-12: 48px;
0027  --xinyu-space-16: 64px;
0028  background: var(--xinyu-color-paper);
0029  color: var(--xinyu-color-text);
0030  font-family: var(--xinyu-font-sans);
0031  font-size: 16px;
0032  line-height: 1.625;
0033 }
0035 /* Main pages share this layout directly; it cannot live only in a component stylesheet. */
0036 .page-container {
0037   min-height: 100vh;
0038   box-sizing: border-box;
0039   background: var(--xinyu-color-paper);
0040   color: var(--xinyu-color-text);
0041 }
0043 .page-container--padded {
0044   padding: 24px 20px calc(88px + env(safe-area-inset-bottom));
0045 }
0047 /* The native tab bar already reserves its own space below the page viewport. */
0048 .page-container--main {
0049   max-width: 480px;
0050   margin: 0 auto;
0051   padding-top: 20px;
0052   padding-bottom: 32px;
0053 }
0055 @media (min-width: 393px) {
0056   .page-container--padded {
0057     padding-left: 24px;
0058     padding-right: 24px;
0059   }
```

miniprogram/app.wxss

```
0060 }
0062 @media (max-width: 359px) {
0063   .page-container--padded {
0064     padding-left: 16px;
0065     padding-right: 16px;
0066   }
0067 }
0069 .page-heading {
0070   margin-bottom: 24px;
0071 }
0072 .page-heading__date {
0073   display: block;
0074   color: var(--xinyu-color-text-secondary);
0075   font-size: 14px;
0076   line-height: 22px;
0077 }
0078 .page-heading__title {
0079   display: block;
0080   margin-top: 2px;
0081   color: var(--xinyu-color-text);
0082   font-size: 28px;
0083   line-height: 38px;
0084   font-weight: 600;
0085 }
0086 .page-heading__greeting {
0087   display: block;
0088   margin-top: 4px;
0089   color: var(--xinyu-color-text-secondary);
0090   font-size: 16px;
0091   line-height: 26px;
0092 }
0093 .page-intro {
0094   margin: -12px 0 24px;
0095   color: var(--xinyu-color-text-secondary);
0096   font-size: 16px;
0097   line-height: 26px;
0098 }
0099 .page-intro__line {
0100   display: block;
0101 }
0103 .reading-block {
0104   padding: 20px 0;
0105   border-top: 1px solid var(--xinyu-color-divider);
0106 }
0107 .reading-block:last-of-type {
0108   border-bottom: 1px solid var(--xinyu-color-divider);
0109 }
0110 .reading-block__label {
0111   display: block;
0112   color: var(--xinyu-color-text-secondary);
0113   font-size: 14px;
```

miniprogram/app.wxss

```
0114     line-height: 22px;
0115 }
0116 .reading-block__title {
0117     display: block;
0118     margin-top: 4px;
0119     color: var(--xinyu-color-text);
0120     font-size: 20px;
0121     line-height: 30px;
0122     font-weight: 500;
0123 }
0124 .reading-block__body {
0125     display: block;
0126     margin-top: 8px;
0127     color: var(--xinyu-color-text-secondary);
0128     font-size: 16px;
0129     line-height: 26px;
0130 }
0131 .reading-block__body--strong {
0132     color: var(--xinyu-color-text);
0133 }
0134 .content-section {
0135     margin-top: 32px;
0136 }
0137 .content-section__title {
0138     display: block;
0139     margin-bottom: 12px;
0140     color: var(--xinyu-color-text);
0141     font-size: 18px;
0142     line-height: 28px;
0143     font-weight: 500;
0144 }
0145 .content-section__hint {
0146     display: block;
0147     margin-top: 8px;
0148     color: var(--xinyu-color-text-secondary);
0149     font-size: 14px;
0150     line-height: 22px;
0151 }
0153 .plain-list {
0154     border-top: 1px solid var(--xinyu-color-divider);
0155 }
0156 .plain-list__item {
0157     display: flex;
0158     align-items: center;
0159     justify-content: space-between;
0160     min-height: 60px;
0161     padding: 12px 0;
0162     box-sizing: border-box;
0163     border-bottom: 1px solid var(--xinyu-color-divider);
0164 }
0165 .plain-list__item > view {
```

miniprogram/app.wxss

```
0166     min-width: 0;
0167     flex: 1;
0168     overflow-wrap: break-word;
0169 }
0170 .section-heading {
0171     display: flex;
0172     align-items: center;
0173     justify-content: space-between;
0174     margin-bottom: 12px;
0175 }
0176 .section-heading .content-section__title {
0177     margin-bottom: 0;
0178 }
0179 .section-heading__action {
0180     flex: 0 0 auto;
0181     min-height: 44px;
0182     margin: 0 0 0 12px;
0183     padding: 8px 0 8px 8px;
0184     background: transparent;
0185     color: var(--xinyu-color-primary);
0186     font-size: 14px;
0187     line-height: 28px;
0188 }
0189 .section-heading__action::after {
0190     border: 0;
0191 }
0192 .plain-list__item--stacked {
0193     display: block;
0194     min-height: 76px;
0195 }
0196 .plain-list__title {
0197     display: block;
0198     color: var(--xinyu-color-text);
0199     font-size: 16px;
0200     line-height: 26px;
0201 }
0202 .plain-list__meta {
0203     display: block;
0204     margin-top: 2px;
0205     color: var(--xinyu-color-text-secondary);
0206     font-size: 14px;
0207     line-height: 22px;
0208 }
0209 .plain-list__status {
0210     display: block;
0211     margin-top: 4px;
0212     color: var(--xinyu-color-text-muted);
0213     font-size: 14px;
0214     line-height: 22px;
0215 }
0216 .plain-list__chevron {
```

miniprogram/app.wxss

```
0217     flex: 0 0 auto;
0218     margin-left: 16px;
0219     color: var(--xinyu-color-text-secondary);
0220     font-size: 22px;
0221     line-height: 22px;
0222 }

0224 .inline-error {
0225     margin-top: 16px;
0226     padding: 12px 0;
0227     border-top: 1px solid var(--xinyu-color-divider);
0228     color: var(--xinyu-color-text-secondary);
0229     font-size: 14px;
0230     line-height: 22px;
0231 }

0232 .inline-error__action {
0233     min-height: 44px;
0234     margin: 4px 0 0;
0235     padding: 8px 0;
0236     border: 0;
0237     background: transparent;
0238     color: var(--xinyu-color-primary);
0239     font-size: 14px;
0240     line-height: 28px;
0241     text-align: left;
0242     text-decoration: underline;
0243 }

0244 .inline-error__action::after {
0245     border: 0;
0246 }

0248 .page-shell {
0249     min-height: 100vh;
0250     padding: var(--xinyu-space-8) var(--xinyu-space-5)
0251         calc(var(--xinyu-space-16) + constant(safe-area-inset-bottom));
0252     padding-bottom: calc(var(--xinyu-space-16) + env(safe-area-inset-bottom));
0253 }

0255 .page-shell__title {
0256     margin: 0;
0257     color: var(--xinyu-color-text);
0258     font-size: 24px;
0259     font-weight: 600;
0260     line-height: 34px;
0261 }

0263 .page-shell__description {
0264     margin: var(--xinyu-space-3) 0 0;
0265     color: var(--xinyu-color-text-secondary);
0266     font-size: 16px;
0267     line-height: 26px;
0268 }

0270 .section-title {
0271     margin: 24px 0 12px;
0272     color: var(--xinyu-color-text);
```

miniprogram/app.wxss

```
0273     font-size: 18px;
0274     line-height: 28px;
0275     font-weight: 500;
0276 }
0277 .section-title--first {
0278     margin-top: 0;
0279 }
0280 .page-shell__description--first {
0281     margin-top: 0;
0282 }
0283 .page-shell__title--compact {
0284     font-size: 20px;
0285     line-height: 30px;
0286 }
0287 .muted {
0288     color: var(--xinyu-color-text-secondary);
0289     font-size: 14px;
0290     line-height: 22px;
0291 }
0292 .surface {
0293     padding: 20px;
0294     background: var(--xinyu-color-surface);
0295     border: 1px solid var(--xinyu-color-divider);
0296     border-radius: 8px;
0297 }
0298 .surface + .surface {
0299     margin-top: 12px;
0300 }
0301 .link-row {
0302     display: flex;
0303     align-items: center;
0304     justify-content: space-between;
0305     min-height: 52px;
0306     border-bottom: 1px solid var(--xinyu-color-divider);
0307     color: var(--xinyu-color-text);
0308 }
0309 .link-row:last-child {
0310     border-bottom: 0;
0311 }
0312 .status-chip {
0313     display: inline-block;
0314     padding: 4px 8px;
0315     border-radius: 4px;
0316     background: var(--xinyu-color-primary-soft);
0317     color: var(--xinyu-color-primary-pressed);
0318     font-size: 12px;
0319     line-height: 18px;
0320 }
0321 .status-chip--waiting {
0322     background: var(--xinyu-color-caution-soft);
0323     color: var(--xinyu-color-caution);
```

miniprogram/app.wxss

```
0324 }
0325 .status-chip--safety {
0326   background: var(--xinyu-color-safety-soft);
0327   color: var(--xinyu-color-safety);
0328 }
0329 .textarea {
0330   width: 100%;
0331   min-height: 180px;
0332   box-sizing: border-box;
0333   padding: 12px;
0334   background: var(--xinyu-color-surface);
0335   border: 1px solid var(--xinyu-color-divider);
0336   border-radius: 8px;
0337   font-size: 16px;
0338   line-height: 26px;
0339 }
0340 .input {
0341   width: 100%;
0342   box-sizing: border-box;
0343   height: 48px;
0344   margin-top: 8px;
0345   padding: 0 12px;
0346   background: var(--xinyu-color-surface);
0347   border: 1px solid var(--xinyu-color-divider);
0348   border-radius: 8px;
0349   font-size: 16px;
0350 }
0351 .option {
0352   display: flex;
0353   align-items: center;
0354   min-height: 52px;
0355   margin-top: 8px;
0356   padding: 0 16px;
0357   box-sizing: border-box;
0358   background: var(--xinyu-color-surface);
0359   border: 1px solid var(--xinyu-color-divider);
0360   border-radius: 8px;
0361   color: var(--xinyu-color-text);
0362 }
0363 .option--selected {
0364   border: 2px solid var(--xinyu-color-primary);
0365   background: var(--xinyu-color-primary-soft);
0366   color: var(--xinyu-color-primary-pressed);
0367 }
0368 .progress {
0369   height: 4px;
0370   margin: 16px 0 24px;
0371   background: var(--xinyu-color-divider);
0372   border-radius: 4px;
0373   overflow: hidden;
0374 }
```

backend/app/services/safety_service.py

```
0044 )
0045 from app.schemas.errors import ApiException
0046 from app.security.tokens import TokenManager
0047 from app.services.idempotency_service import (
0048     IdempotencyReservation,
0049     IdempotencyService,
0050     deserialize_api_error,
0051     serialize_api_error,
0052 )
0054 logger = logging.getLogger(__name__)
0057 class SafetyService:
0058     def __init__(
0059         self,
0060         *,
0061         settings: Settings,
0062         repository: InMemoryDomainDataRepository,
0063         session_repository: InMemorySessionRepository,
0064         token_manager: TokenManager,
0065         idempotency_service: IdempotencyService,
0066         audit_writer: AuditWriter,
0067     ) -> None:
0068         self.settings = settings
0069         self.repository = repository
0070         self.sessions = session_repository
0071         self.tokens = token_manager
0072         self.idempotency = idempotency_service
0073         self.audit = audit_writer
0075     def confirm_safety(
0076         self,
0077         access_token: str,
0078         *,
0079         session_id: str,
0080         state: SafetyConfirmationState,
0081         object_version: int,
0082         answers: list[dict[str, object]],
0083         request_id: str,
0084         idempotency_key: str,
0085     ) -> SafetyConfirmationResponse:
0086         try:
0087             request = SafetyConfirmationRequest.model_validate(
0088                 {"state": state, "answers": answers, "object_version": object_version}
0089             )
0090         except ValidationError as error:
0091             raise ApiException(422, "VALIDATION_FAILED") from error
0092         subject = self.tokens.authenticate_access(access_token, self.sessions)
0093         if subject.subject_type != "student":
0094             raise ApiException(403, "FORBIDDEN")
0095         self._ensure_assessment_access(subject.subject_id)
0096         reservation = self.idempotency.begin(
0097             "student",
```

backend/app/services/safety_service.py

```
0098         subject.subject_id,
0099         f"/assessment-sessions/{session_id}/safety-confirmation",
0100         idempotency_key,
0101         request.model_dump(mode="json"),
0102     )
0103     if reservation.replayed:
0104         return self._decode_confirmation_response(reservation.record.response_digest)
0105
0106     try:
0107         response = self._confirm_safety(
0108             subject_id=subject.subject_id,
0109             session_id=session_id,
0110             request=request,
0111             request_id=request_id,
0112         )
0113         self.idempotency.complete(
0114             reservation,
0115             status_code=200,
0116             response_digest=response.model_dump_json(),
0117         )
0118         return response
0119     except RepositoryVersionConflict as error:
0120         failure = ApiException(409, "VERSION_CONFLICT",
current_version=error.current_version)
0121         _complete_failure(self.idempotency, reservation, failure)
0122         raise failure from error
0123     except RepositoryError as error:
0124         failure = _repository_failure(error)
0125         _complete_failure(self.idempotency, reservation, failure)
0126         raise failure from error
0127     except ApiException as error:
0128         _complete_failure(self.idempotency, reservation, error)
0129         raise
0130     except Exception as error:
0131         failure = ApiException(500, "INTERNAL_ERROR")
0132         _complete_failure(self.idempotency, reservation, failure)
0133         raise failure from error
0134
0135     def acknowledge_support_resource(
0136         self,
0137         access_token: str,
0138         *,
0139         session_id: str,
0140         resource_context: str,
0141         resource_version: str,
0142         object_version: int,
0143         request_id: str,
0144         idempotency_key: str,
0145     ) -> SupportResourceAckResponse:
0146         try:
0147             request = SupportResourceAckRequest.model_validate(
0148                 {
0149                     "resource_context": resource_context,
```

backend/app/services/safety_service.py

```
0150         "resource_version": resource_version,
0151         "object_version": object_version,
0152     }
0153 )
0154 except ValidationError as error:
0155     raise ApiException(422, "VALIDATION_FAILED") from error
0156 subject = self.tokens.authenticate_access(access_token, self.sessions)
0157 if subject.subject_type != "student":
0158     raise ApiException(403, "FORBIDDEN")
0159 self._ensure_assessment_access(subject.subject_id)
0160 reservation = self.idempotency.begin(
0161     "student",
0162     subject.subject_id,
0163     f"/assessment-sessions/{session_id}/support-resource-ack",
0164     idempotency_key,
0165     request.model_dump(mode="json"),
0166 )
0167 if reservation.replayed:
0168     return self._decode_ack_response(reservation.record.response_digest)
0169
0170 try:
0171     response = self._acknowledge_support_resource(
0172         subject_id=subject.subject_id,
0173         session_id=session_id,
0174         request=request,
0175         request_id=request_id,
0176     )
0177     self.idempotency.complete(
0178         reservation,
0179         status_code=200,
0180         response_digest=response.model_dump_json(),
0181     )
0182     return response
0183 except RepositoryVersionConflict as error:
0184     failure = ApiException(409, "VERSION_CONFLICT",
0185 current_version=error.current_version)
0186     _complete_failure(self.idempotency, reservation, failure)
0187     raise failure from error
0188 except RepositoryError as error:
0189     failure = _repository_failure(error)
0190     _complete_failure(self.idempotency, reservation, failure)
0191     raise failure from error
0192 except ApiException as error:
0193     _complete_failure(self.idempotency, reservation, error)
0194     raise
0195 except Exception as error:
0196     failure = ApiException(500, "INTERNAL_ERROR")
0197     _complete_failure(self.idempotency, reservation, failure)
0198     raise failure from error
0199
0200 def _confirm_safety(
0201     self,
```

backend/app/services/safety_service.py

```
0202         subject_id: str,
0203         session_id: str,
0204         request: SafetyConfirmationRequest,
0205         request_id: str,
0206     ) -> SafetyConfirmationResponse:
0207         current = datetime.now(UTC)
0208         decision = decide_safety_branch(request.state)
0209         with self.repository.transaction():
0210             session = self.repository.get_assessment_session(session_id)
0211             if session.user_id != subject_id:
0212                 raise ApiException(404, "NOT_FOUND")
0213             if session.version != request.object_version:
0214                 raise ApiException(409, "VERSION_CONFLICT", current_version=session.version)
0215             if session.state != "in_progress":
0216                 raise ApiException(404, "NOT_FOUND")
0217             if session.safety_confirmation_state == "uncertain" and request.state != "uncertain":
0218                 raise ApiException(422, "SAFETY_SUPPORT_BLOCKED")
0219             if current > session.expires_at:
0220                 expired = session.model_copy(update={"state": "expired", "updated_at": current})
0221                 self.repository.save_assessment_session(expired,
0222                 expected_version=session.version)
0223                 raise ApiException(404, "NOT_FOUND")
0224             module = self.repository.get_assessment_module(session.module_code)
0225             questionnaire = self.repository.get_assessment_questionnaire(
0226                 session.module_code,
0227                 session.questionnaire_version,
0228             )
0229             self._validate_safety_answers(module, questionnaire, request.answers)
0230             resource_categories = self._resource_categories()
0231             task_created = False
0232             task_state: str | None = None
0233             update: dict[str, object] = {
0234                 "safety_triggered": True,
0235                 "safety_confirmation_state": request.state,
0236                 "updated_at": current,
0237             }
0238             if request.state == "cannot_be_safe":
0239                 update.update({"state": "abandoned", "abandoned_at": current})
0240             saved_session = self.repository.save_assessment_session(
0241                 session.model_copy(update=update),
0242                 expected_version=session.version,
0243             )
0244             if decision.create_immediate_task:
0245                 task_created, task_state = self._create_safety_tasks(
0246                     user_id=subject_id,
0247                     source_session_id=saved_session.document_id,
0248                     safety_fact=request.state,
0249                     now=current,
0250                 )
0251             projection = minimal_visible_projection(
0252                 state=request.state,
```

backend/app/services/safety_service.py

```
0254         resource_categories_shown=resource_categories
0255         if request.state != "can_be_safe"
0256         else [],
0257         questionnaire_completed=False,
0258         task_state=task_state,
0259     )
0261     self._audit(
0262         request_id=request_id,
0263         actor_id=subject_id,
0264         action="assessment_safety_confirmation",
0265         resource_id=session_id,
0266         reason_code=request.state,
0267         facts={
0268             "action_code": "safety_confirmation",
0269             "object_version": saved_session.version,
0270             "resource_version": self.settings.support_resource_version,
0271             "status": request.state,
0272             "task_kind": "safety_support" if task_created else None,
0273         },
0274     )
0275     return SafetyConfirmationResponse(
0276         next_step=decision.next_step,
0277         result_id=None,
0278         support_required=decision.support_required,
0279         task_created=task_created,
0280         visible_projection=projection,
0281         version=saved_session.version,
0282     )
0284     def _acknowledge_support_resource(
0285         self,
0286         *,
0287         subject_id: str,
0288         session_id: str,
0289         request: SupportResourceAckRequest,
0290         request_id: str,
0291     ) -> SupportResourceAckResponse:
0292         if self.settings.support_resource_version is None:
0293             raise ApiException(503, "DEPENDENCY_UNAVAILABLE")
0294         if request.resource_version != self.settings.support_resource_version:
0295             raise ApiException(
0296                 409,
0297                 "VERSION_CONFLICT",
0298                 current_version=self.settings.support_resource_version,
0299             )
0300         current = datetime.now(UTC)
0301         with self.repository.transaction():
0302             session = self.repository.get_assessment_session(session_id)
0303             if session.user_id != subject_id:
0304                 raise ApiException(404, "NOT_FOUND")
0305             if session.version != request.object_version:
```

backend/app/services/safety_service.py

```
0306         raise ApiException(409, "VERSION_CONFLICT", current_version=session.version)
0307     if session.state != "in_progress" or session.safety_confirmation_state !=
    "uncertain":
0308         raise ApiException(422, "SAFETY_SUPPORT_BLOCKED")
0309     saved_session = self.repository.save_assessment_session(
0310         session.model_copy(
0311             update={
0312                 "safety_resource_version": request.resource_version,
0313                 "safety_resource_acknowledged_at": current,
0314                 "updated_at": current,
0315             }
0316         ),
0317         expected_version=session.version,
0318     )
0319     self._audit(
0320         request_id=request_id,
0321         actor_id=subject_id,
0322         action="assessment_support_resource_ack",
0323         resource_id=session_id,
0324         reason_code="support_resource_acknowledged",
0325         facts={
0326             "action_code": "support_resource_ack",
0327             "object_version": saved_session.version,
0328             "resource_version": request.resource_version,
0329             "status": "acknowledged",
0330         },
0331     )
0332     return SupportResourceAckResponse(
0333         resource_version=request.resource_version,
0334         acknowledged_at=current,
0335         version=saved_session.version,
0336     )
0337
0338 def _validate_safety_answers(
0339     self,
0340     module: AssessmentModuleDocument,
0341     questionnaire: AssessmentQuestionnaireDocument,
0342     answers: Sequence[AssessmentAnswerSubmission],
0343 ) -> None:
0344     if module.module_code != "phq9" or questionnaire.module_code != "phq9":
0345         raise ApiException(422, "VALIDATION_FAILED")
0346     if questionnaire.questionnaire_version != module.current_questionnaire_version:
0347         raise ApiException(409, "VERSION_CONFLICT", current_version=module.version)
0348     try:
0349         questions = questionnaire_questions(questionnaire)
0350     except AssessmentValidationError as error:
0351         raise ApiException(404, "NOT_FOUND") from error
0352     expected_keys = tuple(f"q{number}" for number in range(1, 10))
0353     submitted = [(answer.question_key, answer.option_key) for answer in answers]
0354     if tuple(question_key for question_key, _ in submitted) != expected_keys:
0355         raise ApiException(422, "VALIDATION_FAILED")
0356     option_scores = {
```

backend/app/services/safety_service.py

```
0357         question.question_key: {option.option_key: option.score for option in
question.options}
0358     for question in questions
0359 }
0360 for question_key, option_key in submitted:
0361     if option_key not in option_scores.get(question_key, {}):
0362         raise ApiException(422, "VALIDATION_FAILED")
0363     if option_scores[PHQ9_Q9_KEY][submitted[-1][1]] == 0:
0364         raise ApiException(422, "VALIDATION_FAILED")
0365
0366 def _resource_categories(self) -> list[str]:
0367     if self.settings.environment_kind not in {EnvironmentKind.DEMO,
EnvironmentKind.AUTHORIZED}:
0368         return []
0369     resources = self.repository.list_support_resources(
0370         self.settings.environment_kind.value,
0371         resource_set_version=self.settings.support_resource_version,
0372         now=datetime.now(UTC),
0373     )
0374     return sorted({resource.category for resource in resources})
0375
0376 def _create_safety_tasks(
0377     self,
0378     *,
0379     user_id: str,
0380     source_session_id: str,
0381     safety_fact: str,
0382     now: datetime,
0383 ) -> tuple[bool, str | None]:
0384     if self.settings.environment_kind not in {EnvironmentKind.DEMO,
EnvironmentKind.AUTHORIZED}:
0385         return False, None
0386     resources = self.repository.list_support_resources(
0387         self.settings.environment_kind.value,
0388         resource_set_version=self.settings.support_resource_version,
0389         now=now,
0390     )
0391     snapshot = [
0392         SupportResourceSnapshotModel(
0393             resource_id=resource.document_id,
0394             title=resource.title,
0395             category=resource.category,
0396             action_type=resource.action_type,
0397             action_target=resource.action_target,
0398         )
0399         for resource in resources
0400     ]
0401     safety_task_id = self.repository.next_safety_support_task_id()
0402     safety_task = SafetySupportTaskDocument(
0403         _id=safety_task_id,
0404         task_kind="safety_support",
0405         user_reference_id=user_id,
0406         source_result_id=None,
0407         source_session_id=source_session_id,
0408         safety_fact=safety_fact, # type: ignore[arg-type]
```

backend/app/services/safety_service.py

```
0409         support_resource_snapshot=snapshot,
0410         state="needs_action",
0411         assigned_admin_id=None,
0412         followup_due_at=None,
0413         fact_note=None,
0414         completed_at=None,
0415         created_at=now,
0416         updated_at=now,
0417         version=1,
0418     )
0419     self.repository.create_safety_support_task(safety_task)
0420     work_task = WorkTaskDocument(
0421         _id=safety_task_id,
0422         task_kind="safety_support",
0423         source_type="support_task",
0424         source_id=safety_task.document_id,
0425         available_capability="safety_support",
0426         state="needs_action",
0427         assigned_admin_id=None,
0428         safe_summary="安全支持任务待处理",
0429         object_version=safety_task.version,
0430         last_action=None,
0431         created_at=now,
0432         updated_at=now,
0433         version=1,
0434     )
0435     self.repository.create_work_task(work_task)
0436     return True, safety_task.state
0437
0438 def _ensure_assessment_access(self, user_id: str) -> None:
0439     user = self.repository.get_user(user_id)
0440     if user.status != "active":
0441         raise ApiException(403, "FORBIDDEN")
0442     if user.base_consent_status != "accepted":
0443         raise ApiException(403, "CONSENT_REQUIRED")
0444     if not user.identity_record_id:
0445         raise ApiException(403, "IDENTITY_REQUIRED")
0446     try:
0447         identity = self.repository.get_identity_record(user.identity_record_id)
0448     except RepositoryNotFound as error:
0449         raise ApiException(403, "IDENTITY_REQUIRED") from error
0450     if identity.user_id != user.document_id or identity.verification_status != "verified":
0451         raise ApiException(403, "IDENTITY_REQUIRED")
0452
0453 def _audit(
0454     self,
0455     *,
0456     request_id: str,
0457     actor_id: str,
0458     action: str,
0459     resource_id: str,
0460     reason_code: str,
```

backend/app/services/safety_service.py

```
0461         facts: dict[str, object],
0462     ) -> None:
0463         try:
0464             self.audit.write(
0465                 request_id=request_id,
0466                 actor_type="student",
0467                 actor_id=actor_id,
0468                 capability=None,
0469                 action=action,
0470                 resource_type="assessment",
0471                 resource_id=resource_id,
0472                 data_scope="necessary_facts",
0473                 outcome="success",
0474                 reason_code=reason_code,
0475                 occurred_at=datetime.now(UTC),
0476                 facts=facts,
0477             )
0478         except Exception:
0479             logger.warning("audit_write_failed")
0481     def _decode_confirmation_response(
0482         self, response_digest: str | None
0483     ) -> SafetyConfirmationResponse:
0484         if response_digest is None:
0485             raise ApiException(500, "INTERNAL_ERROR")
0486         error = deserialize_api_error(response_digest)
0487         if error is not None:
0488             raise error
0489         return SafetyConfirmationResponse.model_validate_json(response_digest)
0491     def _decode_ack_response(self, response_digest: str | None) -> SupportResourceAckResponse:
0492         if response_digest is None:
0493             raise ApiException(500, "INTERNAL_ERROR")
0494         error = deserialize_api_error(response_digest)
0495         if error is not None:
0496             raise error
0497         return SupportResourceAckResponse.model_validate_json(response_digest)
0500     def _complete_failure(
0501         idempotency: IdempotencyService,
0502         reservation: IdempotencyReservation,
0503         error: ApiException,
0504     ) -> None:
0505         idempotency.complete(
0506             reservation,
0507             status_code=error.status_code,
0508             response_digest=serialize_api_error(error),
0509             outcome="failure",
0510         )
0513     def _repository_failure(error: RepositoryError) -> ApiException:
0514         if isinstance(error, RepositoryNotFound):
0515             return ApiException(404, "NOT_FOUND")
0516         return ApiException(503, "DEPENDENCY_UNAVAILABLE")
```

backend/app/services/treehole_service.py

```
0001 from __future__ import annotations
0002 import hashlib
0003 from datetime import UTC, datetime
0004 from typing import Literal
0005
0006 from app.audit.writer import AuditWriter
0007 from app.config.settings import Settings
0008 from app.domain.models import TreeholePostDocument, TreeholeResponseDocument
0009 from app.repositories.domain_data_repository import InMemoryDomainDataRepository
0010 from app.repositories.protocols import (
0011     RepositoryError,
0012     RepositoryNotFound,
0013     RepositoryVersionConflict,
0014 )
0015
0016 from app.repositories.session_repository import InMemorySessionRepository
0017 from app.schemas.errors import ApiException
0018 from app.schemas.treehole import (
0019     TreeholeDeleteResponse,
0020     TreeholeMutationResponse,
0021     TreeholePostListResponse,
0022     TreeholePostProjection,
0023     TreeholeResponseProjection,
0024 )
0025
0026 from app.security.tokens import AuthenticatedSubject, TokenManager
0027 from app.services.consent_service import ConsentService
0028 from app.services.idempotency_service import (
0029     IdempotencyReservation,
0030     IdempotencyService,
0031     deserialize_api_error,
0032     serialize_api_error,
0033 )
0034
0035 class TreeholeService:
0036     def __init__(
0037         self,
0038         *,
0039         settings: Settings,
0040         repository: InMemoryDomainDataRepository,
0041         session_repository: InMemorySessionRepository,
0042         token_manager: TokenManager,
0043         idempotency_service: IdempotencyService,
0044         audit_writer: AuditWriter,
0045         consent_service: ConsentService,
0046     ) -> None:
0047         self.settings = settings
0048         self.repository = repository
0049         self.sessions = session_repository
0050         self.tokens = token_manager
0051         self.idempotency = idempotency_service
0052         self.audit = audit_writer
0053         self.consent = consent_service
0054
0055     def list_public(
```

backend/app/services/treehole_service.py

```
0056         self,
0057         access_token: str,
0058         *,
0059         sort: str = "latest",
0060         cursor: str | None = None,
0061         limit: int = 20,
0062     ) -> TreeholePostListResponse:
0063         subject = self._student(access_token)
0064         if sort not in {"latest", "confirmed"} or limit < 1:
0065             raise ApiException(422, "VALIDATION_FAILED")
0066         posts = [
0067             post
0068             for post in self.repository.list_treehole_posts()
0069             if post.visibility_state in {"published", "protected"} and post.deleted_at is None
0070         ]
0071         if sort == "confirmed":
0072             posts.sort(
0073                 key=lambda post: (post.visibility_state == "protected", post.created_at),
0074                 reverse=True,
0075             )
0076         offset = _decode_cursor(cursor)
0077         page_size = min(limit, 100)
0078         page = posts[offset : offset + page_size]
0079         next_cursor = str(offset + page_size) if offset + page_size < len(posts) else None
0080         return TreeholePostListResponse(
0081             items=[
0082                 self._post_projection(post, viewer_id=subject.subject_id, public=True)
0083                 for post in page
0084             ],
0085             next_cursor=next_cursor,
0086         )
0088     def get_post(self, access_token: str, *, post_id: str) -> TreeholePostProjection:
0089         subject = self._student(access_token)
0090         try:
0091             post = self.repository.get_treehole_post(post_id)
0092         except RepositoryNotFound as error:
0093             raise ApiException(404, "NOT_FOUND") from error
0094         mine = post.author_user_id == subject.subject_id
0095         if not mine and post.visibility_state not in {"published", "protected"}:
0096             raise ApiException(404, "NOT_FOUND")
0097         return self._post_projection(post, viewer_id=subject.subject_id, public=not mine)
0099     def list_mine(
0100         self,
0101         access_token: str,
0102         *,
0103         cursor: str | None = None,
0104         limit: int = 20,
0105     ) -> TreeholePostListResponse:
0106         subject = self._student(access_token)
0107         if limit < 1:
```

backend/app/services/treehole_service.py

```
0108         raise ApiException(422, "VALIDATION_FAILED")
0109     posts = [
0110         post
0111         for post in self.repository.list_treehole_posts()
0112         if post.author_user_id == subject.subject_id
0113     ]
0114     offset = _decode_cursor(cursor)
0115     page_size = min(limit, 100)
0116     page = posts[offset : offset + page_size]
0117     next_cursor = str(offset + page_size) if offset + page_size < len(posts) else None
0118     return TreeholePostListResponse(
0119         items=[
0120             self._post_projection(post, viewer_id=subject.subject_id, public=False)
0121             for post in page
0122         ],
0123         next_cursor=next_cursor,
0124     )
0126     def create_post(
0127         self,
0128         access_token: str,
0129         *,
0130         body: str,
0131         request_id: str,
0132         idempotency_key: str,
0133     ) -> TreeholeMutationResponse:
0134         if not body.strip():
0135             raise ApiException(422, "VALIDATION_FAILED")
0136         subject = self._student(access_token)
0137         self.consent.ensure_community_write_allowed(access_token)
0138         reservation = self.idempotency.begin(
0139             "student", subject.subject_id, "/treehole/posts", idempotency_key, {"body": body}
0140         )
0141         if reservation.replayed:
0142             return _decode_mutation(reservation.record.response_digest)
0143         try:
0144             user = self.repository.get_user(subject.subject_id)
0145             if not user.anonymous_identity_id:
0146                 raise ApiException(403, "IDENTITY_REQUIRED")
0147             anonymous = self.repository.get_anonymous_identity(user.anonymous_identity_id)
0148             if anonymous.user_id != user.document_id or anonymous.status != "active":
0149                 raise ApiException(403, "IDENTITY_REQUIRED")
0150             now = datetime.now(UTC)
0151             safety = _contains_safety_signal(body)
0152             state: Literal[
0153                 "checking",
0154                 "published",
0155                 "protected",
0156                 "pending_confirmation",
0157                 "unpublished",
0158                 "safety_priority",
```

backend/app/services/treehole_service.py

```
0159         "deleted",
0160     ] = "safety_priority" if safety else "checking"
0161     review_state: Literal[
0162         "not_started", "automated_checked", "human_required", "decided"
0163     ] = "human_required"
0164     post = TreeholePostDocument(
0165         _id=self.repository.next_treehole_post_id(),
0166         author_user_id=user.document_id,
0167         anonymous_identity_id=anonymous.document_id,
0168         display_name_snapshot=anonymous.display_name,
0169         body_original_ciphertext=_protected_body(body),
0170         body_sanitized=None,
0171         body_hash=hashlib.sha256(body.encode("utf-8")).hexdigest(),
0172         visibility_state=state,
0173         review_state=review_state,
0174         safety_state="needs_support_review" if safety else "not_triggered",
0175         community_consent_version=user.community_consent_version or "unknown",
0176         original_retention_deadline=None,
0177         deleted_at=None,
0178         created_at=now,
0179         updated_at=now,
0180         version=1,
0181     )
0182     with self.repository.transaction():
0183         saved = self.repository.create_treehole_post(post)
0184         self.repository.append_extra_document(
0185             "content_review_tasks",
0186             {
0187                 "_id": f"content_review_{saved.document_id}",
0188                 "post_id": saved.document_id,
0189                 "state": "needs_action",
0190                 "safety_priority": safety,
0191             },
0192         )
0193     response = self._mutation(saved, subject.subject_id)
0194     self.idempotency.complete(
0195         reservation, status_code=200, response_digest=response.model_dump_json()
0196     )
0197     self._audit(
0198         request_id, subject.subject_id, "treehole_post_create", saved.document_id, state
0199     )
0200     return response
0201 except RepositoryVersionConflict as error:
0202     failure = ApiException(409, "VERSION_CONFLICT",
current_version=error.current_version)
0203     _complete_failure(self.idempotency, reservation, failure)
0204     raise failure from error
0205 except RepositoryError as error:
0206     failure = ApiException(503, "DEPENDENCY_UNAVAILABLE")
0207     _complete_failure(self.idempotency, reservation, failure)
0208     raise failure from error
```

backend/app/services/treehole_service.py

```
0209         except ApiException as error:
0210             _complete_failure(self.idempotency, reservation, error)
0211             raise
0212         except Exception as error:
0213             failure = ApiException(500, "INTERNAL_ERROR")
0214             _complete_failure(self.idempotency, reservation, failure)
0215             raise failure from error
0217     def withdraw_post(
0218         self,
0219         access_token: str,
0220         *,
0221         post_id: str,
0222         object_version: int,
0223         request_id: str,
0224         idempotency_key: str,
0225     ) -> TreeholeMutationResponse:
0226         return self._change_post_state(
0227             access_token,
0228             post_id=post_id,
0229             object_version=object_version,
0230             target="unpublished",
0231             request_id=request_id,
0232             idempotency_key=idempotency_key,
0233         )
0235     def delete_post(
0236         self,
0237         access_token: str,
0238         *,
0239         post_id: str,
0240         object_version: int,
0241         request_id: str,
0242         idempotency_key: str,
0243     ) -> TreeholeDeleteResponse:
0244         subject = self._student(access_token)
0245         reservation = self.idempotency.begin(
0246             "student",
0247             subject.subject_id,
0248             f"/treehole/posts/{post_id}/delete",
0249             idempotency_key,
0250             {"post_id": post_id, "object_version": object_version},
0251         )
0252         if reservation.replayed:
0253             return _decode_delete(reservation.record.response_digest)
0254         try:
0255             with self.repository.transaction():
0256                 post = self.repository.get_treehole_post(post_id)
0257                 self._check_owner(post.author_user_id, subject.subject_id)
0258                 if post.version != object_version:
0259                     raise ApiException(409, "VERSION_CONFLICT", current_version=post.version)
0260                 if post.deleted_at is not None:
```

backend/app/services/treehole_service.py

```
0261         raise ApiException(404, "NOT_FOUND")
0262         now = datetime.now(UTC)
0263         saved = self.repository.save_treehole_post(
0264             post.model_copy(
0265                 update={
0266                     "visibility_state": "deleted",
0267                     "deleted_at": now,
0268                     "body_sanitized": None,
0269                 }
0270             ),
0271             expected_version=post.version,
0272         )
0273         response = TreeholeDeleteResponse(
0274             resource_id=saved.document_id,
0275             deleted_at=saved.deleted_at or now,
0276             object_version=saved.version,
0277         )
0278         self.idempotency.complete(
0279             reservation, status_code=200, response_digest=response.model_dump_json()
0280         )
0281         self._audit(request_id, subject.subject_id, "treehole_post_delete", post_id,
0282 "deleted")
0283         return response
0284     except RepositoryNotFound as error:
0285         failure = ApiException(404, "NOT_FOUND")
0286         _complete_failure(self.idempotency, reservation, failure)
0287         raise failure from error
0288     except RepositoryVersionConflict as error:
0289         failure = ApiException(409, "VERSION_CONFLICT",
0290 current_version=error.current_version)
0291         _complete_failure(self.idempotency, reservation, failure)
0292         raise failure from error
0293     except ApiException as error:
0294         _complete_failure(self.idempotency, reservation, error)
0295         raise
0296     except Exception as error:
0297         failure = ApiException(500, "INTERNAL_ERROR")
0298         _complete_failure(self.idempotency, reservation, failure)
0299         raise failure from error
0300
0301     def create_response(
0302         self,
0303         access_token: str,
0304         *,
0305         post_id: str,
0306         body: str,
0307         object_version: int,
0308         request_id: str,
0309         idempotency_key: str,
0310     ) -> TreeholeResponseProjection:
0311         subject = self._student(access_token)
0312         self.consent.ensure_community_write_allowed(access_token)
0313         reservation = self.idempotency.begin(
```

backend/app/services/treehole_service.py

```
0312         "student",
0313         subject.subject_id,
0314         f"/treehole/posts/{post_id}/responses",
0315         idempotency_key,
0316         {"post_id": post_id, "body": body, "object_version": object_version},
0317     )
0318     if reservation.replayed:
0319         return _decode_response(reservation.record.response_digest)
0320     try:
0321         post = self.repository.get_treehole_post(post_id)
0322         if (
0323             post.visibility_state not in {"published", "protected"}
0324             or post.deleted_at is not None
0325         ):
0326             raise ApiException(404, "NOT_FOUND")
0327         if post.version != object_version:
0328             raise ApiException(409, "VERSION_CONFLICT", current_version=post.version)
0329         user = self.repository.get_user(subject.subject_id)
0330         if not user.anonymous_identity_id:
0331             raise ApiException(403, "IDENTITY_REQUIRED")
0332         anonymous = self.repository.get_anonymous_identity(user.anonymous_identity_id)
0333         now = datetime.now(UTC)
0334         response = TreeholeResponseDocument(
0335             _id=self.repository.next_treehole_response_id(),
0336             post_id=post.document_id,
0337             author_user_id=user.document_id,
0338             anonymous_identity_id=anonymous.document_id,
0339             display_name_snapshot=anonymous.display_name,
0340             body_original_ciphertext=_protected_body(body),
0341             body_sanitized=None,
0342             state="checking",
0343             community_consent_version=user.community_consent_version or "unknown",
0344             deleted_at=None,
0345             created_at=now,
0346             updated_at=now,
0347             version=1,
0348         )
0349         saved = self.repository.create_treehole_response(response)
0350         projection = _response_projection(saved, viewer_id=subject.subject_id, public=False)
0351         self.idempotency.complete(
0352             reservation, status_code=200, response_digest=projection.model_dump_json()
0353         )
0354         self._audit(
0355             request_id,
0356             subject.subject_id,
0357             "treehole_response_create",
0358             saved.document_id,
0359             "checking",
0360         )
0361     return projection
```

backend/app/services/treehole_service.py

```
0362         except RepositoryNotFound as error:
0363             failure = ApiException(404, "NOT_FOUND")
0364             _complete_failure(self.idempotency, reservation, failure)
0365             raise failure from error
0366         except ApiException as error:
0367             _complete_failure(self.idempotency, reservation, error)
0368             raise
0369         except Exception as error:
0370             failure = ApiException(500, "INTERNAL_ERROR")
0371             _complete_failure(self.idempotency, reservation, failure)
0372             raise failure from error
0373
0374     def delete_response(
0375         self,
0376         access_token: str,
0377         *,
0378         response_id: str,
0379         object_version: int,
0380         request_id: str,
0381         idempotency_key: str,
0382     ) -> TreeholeDeleteResponse:
0383         subject = self._student(access_token)
0384         reservation = self.idempotency.begin(
0385             "student",
0386             subject.subject_id,
0387             f"/treehole/responses/{response_id}",
0388             idempotency_key,
0389             {"response_id": response_id, "object_version": object_version},
0390         )
0391         if reservation.replayed:
0392             return _decode_delete(reservation.record.response_digest)
0393         try:
0394             response = self.repository.get_treehole_response(response_id)
0395             self._check_owner(response.author_user_id, subject.subject_id)
0396             if response.version != object_version:
0397                 raise ApiException(409, "VERSION_CONFLICT", current_version=response.version)
0398             if response.deleted_at is not None:
0399                 raise ApiException(404, "NOT_FOUND")
0400             now = datetime.now(UTC)
0401             saved = self.repository.save_treehole_response(
0402                 response.model_copy(
0403                     update={"state": "deleted", "deleted_at": now, "body_sanitized": None}
0404                 ),
0405                 expected_version=response.version,
0406             )
0407             result = TreeholeDeleteResponse(
0408                 resource_id=saved.document_id,
0409                 deleted_at=saved.deleted_at or now,
0410                 object_version=saved.version,
0411             )
0412             self.idempotency.complete(
```

backend/app/services/treehole_service.py

```
0413         reservation, status_code=200, response_digest=result.model_dump_json()
0414     )
0415     self._audit(
0416         request_id, subject.subject_id, "treehole_response_delete", response_id,
    "deleted"
0417     )
0418     return result
0419 except RepositoryNotFound as error:
0420     failure = ApiException(404, "NOT_FOUND")
0421     _complete_failure(self.idempotency, reservation, failure)
0422     raise failure from error
0423 except RepositoryVersionConflict as error:
0424     failure = ApiException(409, "VERSION_CONFLICT",
current_version=error.current_version)
0425     _complete_failure(self.idempotency, reservation, failure)
0426     raise failure from error
0427 except ApiException as error:
0428     _complete_failure(self.idempotency, reservation, error)
0429     raise
0430 except Exception as error:
0431     failure = ApiException(500, "INTERNAL_ERROR")
0432     _complete_failure(self.idempotency, reservation, failure)
0433     raise failure from error
0435 def _change_post_state(
0436     self,
0437     access_token: str,
0438     *,
0439     post_id: str,
0440     object_version: int,
0441     target: str,
0442     request_id: str,
0443     idempotency_key: str,
0444 ) -> TreeholeMutationResponse:
0445     subject = self._student(access_token)
0446     reservation = self.idempotency.begin(
0447         "student",
0448         subject.subject_id,
0449         f"/treehole/posts/{post_id}/{target}",
0450         idempotency_key,
0451         {"post_id": post_id, "object_version": object_version, "target": target},
0452     )
0453     if reservation.replayed:
0454         return _decode_mutation(reservation.record.response_digest)
0455     try:
0456         with self.repository.transaction():
0457             post = self.repository.get_treehole_post(post_id)
0458             self._check_owner(post.author_user_id, subject.subject_id)
0459             if post.version != object_version:
0460                 raise ApiException(409, "VERSION_CONFLICT", current_version=post.version)
0461             if post.deleted_at is not None:
0462                 raise ApiException(404, "NOT_FOUND")
0463             saved = self.repository.save_treehole_post(
```

backend/app/services/treehole_service.py

```
0464         post.model_copy(update={"visibility_state": target}),
0465         expected_version=post.version,
0466     )
0467     result = self._mutation(saved, subject.subject_id)
0468     self.idempotency.complete(
0469         reservation, status_code=200, response_digest=result.model_dump_json()
0470     )
0471     self._audit(request_id, subject.subject_id, "treehole_post_withdraw", post_id,
target)
0472     return result
0473     except RepositoryNotFound as error:
0474         failure = ApiException(404, "NOT_FOUND")
0475         _complete_failure(self.idempotency, reservation, failure)
0476         raise failure from error
0477     except RepositoryVersionConflict as error:
0478         failure = ApiException(409, "VERSION_CONFLICT",
current_version=error.current_version)
0479         _complete_failure(self.idempotency, reservation, failure)
0480         raise failure from error
0481     except ApiException as error:
0482         _complete_failure(self.idempotency, reservation, error)
0483         raise
0484     except Exception as error:
0485         failure = ApiException(500, "INTERNAL_ERROR")
0486         _complete_failure(self.idempotency, reservation, failure)
0487         raise failure from error
0489     def _student(self, access_token: str) -> AuthenticatedSubject:
0490         subject = self.tokens.authenticate_access(access_token, self.sessions)
0491         if subject.subject_type != "student":
0492             raise ApiException(403, "FORBIDDEN")
0493         return subject
0495     def _post_projection(
0496         self, post: TreeholePostDocument, *, viewer_id: str, public: bool
0497     ) -> TreeholePostProjection:
0498         mine = post.author_user_id == viewer_id
0499         expose_body = post.visibility_state in {"published", "protected"}
0500         if public and not expose_body:
0501             expose_body = False
0502         responses = [
0503             _response_projection(response, viewer_id=viewer_id, public=public)
0504             for response in self.repository.list_treehole_responses(post.document_id)
0505             if response.deleted_at is None and (not public or response.state == "published")
0506         ]
0507         return TreeholePostProjection(
0508             post_id=post.document_id,
0509             display_name_snapshot=post.display_name_snapshot if (public or mine) else None,
0510             body_sanitized=post.body_sanitized if expose_body else None,
0511             visibility_state=post.visibility_state,
0512             review_state=post.review_state,
0513             safety_state=post.safety_state if mine else None,
0514             created_at=post.created_at,
0515             updated_at=post.updated_at,
```

backend/app/services/treehole_service.py

```
0516         response_count=len(responses),
0517         object_version=post.version,
0518         mine=mine,
0519         responses=responses,
0520     )
0521
0522     def _mutation(self, post: TreeholePostDocument, viewer_id: str) -> TreeholeMutationResponse:
0523         projection = self._post_projection(post, viewer_id=viewer_id, public=False)
0524         return TreeholeMutationResponse(
0525             post_id=post.document_id,
0526             visibility_state=post.visibility_state,
0527             review_state=post.review_state,
0528             display_projection=projection,
0529             object_version=post.version,
0530         )
0531
0532     def _check_owner(self, owner_id: str, subject_id: str) -> None:
0533         if owner_id != subject_id:
0534             raise ApiException(404, "NOT_FOUND")
0535
0536     def _audit(
0537         self, request_id: str, actor_id: str, action: str, resource_id: str, status: str
0538     ) -> None:
0539         try:
0540             self.audit.write(
0541                 request_id=request_id,
0542                 actor_type="student",
0543                 actor_id=actor_id,
0544                 capability=None,
0545                 action=action,
0546                 resource_type="post",
0547                 resource_id=resource_id,
0548                 data_scope="necessary_facts",
0549                 outcome="success",
0550                 reason_code=status,
0551                 occurred_at=datetime.now(UTC),
0552                 facts={"action_code": action, "status": status},
0553             )
0554         except Exception:
0555             pass
0556
0557     def _protected_body(body: str) -> str:
0558         return "enc:v1:" + hashlib.sha256(body.encode("utf-8")).hexdigest()
0559
0560     def _contains_safety_signal(body: str) -> bool:
0561         return any(token in body for token in ("自杀", "自残", "不想活", "伤害自己"))
0562
0563     def _decode_cursor(cursor: str | None) -> int:
0564         if cursor is None:
0565             return 0
0566         try:
0567             value = int(cursor)
0568         except (TypeError, ValueError) as error:
0569             raise ApiException(422, "VALIDATION_FAILED") from error
0570         if value < 0:
0571             raise ApiException(422, "VALIDATION_FAILED")
```

backend/app/services/treehole_service.py

```
0575         return value

0578     def _response_projection(
0579         response: TreeholeResponseDocument, *, viewer_id: str, public: bool
0580     ) -> TreeholeResponseProjection:
0581         mine = response.author_user_id == viewer_id
0582         expose = response.state == "published" or mine
0583         return TreeholeResponseProjection(
0584             response_id=response.document_id,
0585             post_id=response.post_id,
0586             display_name_snapshot=response.display_name_snapshot if expose else None,
0587             body_sanitized=response.body_sanitized
0588             if expose and response.state == "published"
0589             else None,
0590             state=response.state,
0591             created_at=response.created_at,
0592             updated_at=response.updated_at,
0593             object_version=response.version,
0594         )

0597     def _decode_mutation(value: str | None) -> TreeholeMutationResponse:
0598         if value is None:
0599             raise ApiException(500, "INTERNAL_ERROR")
0600         error = deserialize_api_error(value)
0601         if error is not None:
0602             raise error
0603         try:
0604             return TreeholeMutationResponse.model_validate_json(value)
0605         except Exception as exc:
0606             raise ApiException(500, "INTERNAL_ERROR") from exc

0609     def _decode_response(value: str | None) -> TreeholeResponseProjection:
0610         if value is None:
0611             raise ApiException(500, "INTERNAL_ERROR")
0612         error = deserialize_api_error(value)
0613         if error is not None:
0614             raise error
0615         try:
0616             return TreeholeResponseProjection.model_validate_json(value)
0617         except Exception as exc:
0618             raise ApiException(500, "INTERNAL_ERROR") from exc

0621     def _decode_delete(value: str | None) -> TreeholeDeleteResponse:
0622         if value is None:
0623             raise ApiException(500, "INTERNAL_ERROR")
0624         error = deserialize_api_error(value)
0625         if error is not None:
0626             raise error
0627         try:
0628             return TreeholeDeleteResponse.model_validate_json(value)
0629         except Exception as exc:
0630             raise ApiException(500, "INTERNAL_ERROR") from exc

0633     def _complete_failure(
0634         idempotency: IdempotencyService,
```

backend/app/services/treehole_service.py

```
0635     reservation: IdempotencyReservation,
0636     error: ApiException,
0637 ) -> None:
0638     idempotency.complete(
0639         reservation,
0640         status_code=error.status_code,
0641         response_digest=serialize_api_error(error),
0642         outcome="failure",
0643     )
```

backend/app/services/admin_workbench_service.py

```
0001 """Server-side projections and state transitions for the admin workbench."""
0002
0003 from __future__ import annotations
0004
0005 from datetime import UTC, datetime, timedelta
0006 from typing import Any, Literal, cast
0007
0008 from app.audit.writer import AuditWriter
0009 from app.config.settings import Settings
0010 from app.repositories.audit_repository import AuditRepository, InMemoryAuditRepository
0011 from app.schemas.admin_workbench import (
0012     AuditEvent,
0013     TaskDetail,
0014     TaskFact,
0015     TaskKind,
0016     TaskMutationResult,
0017     TaskSummary,
0018     WorkbenchPage,
0019     WorkbenchSection,
0020 )
0021 from app.schemas.errors import ApiException
0022
0023 class AdminWorkbenchService:
0024     """Owns the minimal demo task index used by the HTTP adapter.
0025
0026     The service deliberately stores only projected facts. A production deployment
0027     can replace this store with a repository without changing the API projection.
0028     """
0029
0030     def __init__(
0031         self,
0032         settings: Settings,
0033         *,
0034         audit_repository: AuditRepository | None = None,
0035     ) -> None:
0036         self.settings = settings
0037         self.tasks: dict[str, dict[str, Any]] = {}
0038         self.audit = AuditWriter(
0039             audit_repository or InMemoryAuditRepository(),
0040             environment_id=settings.cloudbase_env_id or settings.environment_kind.value,
0041         )
0042         self._idempotency: dict[tuple[str, str], TaskMutationResult] = {}
0043         self._seed_demo_tasks()
0044
0045     def list_tasks(
0046         self,
0047         section: WorkbenchSection,
0048         *,
0049         admin_id: str,
0050         cursor: str | None,
0051         limit: int,
0052     ) -> WorkbenchPage:
0053         tasks = [
```

backend/app/services/admin_workbench_service.py

```
0055         task
0056         for task in self.tasks.values()
0057         if not task.get("is_deleted", False) and self._in_section(task, section, admin_id)
0058     ]
0059     tasks.sort(key=lambda item: str(item["updated_at"]), reverse=True)
0060     offset = self._decode_cursor(cursor)
0061     page_items = tasks[offset : offset + max(1, min(limit, 100))]
0062     next_cursor = (
0063         str(offset + len(page_items)) if offset + len(page_items) < len(tasks) else None
0064     )
0065     return WorkbenchPage(
0066         items=[self._summary(task, admin_id=admin_id) for task in page_items],
0067         next_cursor=next_cursor,
0068     )
0070 def get_task(self, task_id: str, *, admin_id: str) -> TaskDetail:
0071     task = self.tasks.get(task_id)
0072     if task is None or task.get("is_deleted", False):
0073         raise ApiException(404, "NOT_FOUND")
0074     return self._detail(task)
0076 def record_view(self, task_id: str, *, admin_id: str, request_id: str) -> None:
0077     """Record a minimal detail-view audit event after the projection is authorized."""
0079     task = self.tasks.get(task_id)
0080     if task is None or task.get("is_deleted", False):
0081         return
0082     self.audit.write(
0083         request_id=request_id,
0084         actor_type="admin",
0085         actor_id=admin_id,
0086         capability="super_admin",
0087         action="task_view",
0088         resource_type="task",
0089         resource_id=task_id,
0090         data_scope="necessary_facts,redacted_content",
0091         outcome="success",
0092         reason_code=None,
0093         occurred_at=datetime.now(UTC),
0094         facts={"task_kind": task["task_kind"], "object_version": task["version"]},
0095     )
0097 def record_task_outcome(
0098     self,
0099     task_id: str,
0100     *,
0101     admin_id: str,
0102     request_id: str,
0103     action: str,
0104     outcome: Literal["denied", "conflict", "failure"],
0105     reason_code: str,
0106 ) -> None:
0107     task = self.tasks.get(task_id)
0108     if task is None or task.get("is_deleted", False):
0109         return
```

backend/app/services/admin_workbench_service.py

```
0110         self.audit.write(  
0111             request_id=request_id,  
0112             actor_type="admin",  
0113             actor_id=admin_id,  
0114             capability="super_admin",  
0115             action=action,  
0116             resource_type="task",  
0117             resource_id=task_id,  
0118             data_scope="task_state,object_version",  
0119             outcome=outcome,  
0120             reason_code=reason_code,  
0121             occurred_at=datetime.now(UTC),  
0122             facts={"task_kind": task["task_kind"], "object_version": task["version"]},  
0123         )  
0124  
0125     def claim(  
0126         self,  
0127         task_id: str,  
0128         *,  
0129         object_version: int,  
0130         admin_id: str,  
0131         request_id: str,  
0132         idempotency_key: str,  
0133     ) -> TaskMutationResult:  
0134         task = self._get_mutable(task_id, object_version, admin_id, idempotency_key)  
0135         if task["state"] != "needs_action":  
0136             raise ApiException(409, "VERSION_CONFLICT", current_version=task["version"])  
0137         task.update(  
0138             {  
0139                 "state": "claimed",  
0140                 "assigned_admin_id": admin_id,  
0141                 "version": task["version"] + 1,  
0142                 "updated_at": datetime.now(UTC),  
0143             }  
0144         )  
0145         result = self._result(task, request_id)  
0146         self._remember(admin_id, idempotency_key, result)  
0147         self._write_audit(request_id, task, "task_claim", "success")  
0148         return result  
0149  
0150     def release(  
0151         self,  
0152         task_id: str,  
0153         *,  
0154         object_version: int,  
0155         admin_id: str,  
0156         request_id: str,  
0157         idempotency_key: str,  
0158     ) -> TaskMutationResult:  
0159         task = self._get_mutable(task_id, object_version, admin_id, idempotency_key)  
0160         if task.get("assigned_admin_id") != admin_id or task["state"] != "claimed":  
0161             raise ApiException(403, "FORBIDDEN")
```

backend/app/services/admin_workbench_service.py

```
0162         task.update(
0163             {
0164                 "state": "needs_action",
0165                 "assigned_admin_id": None,
0166                 "version": task["version"] + 1,
0167                 "updated_at": datetime.now(UTC),
0168             }
0169         )
0170         result = self._result(task, request_id)
0171         self._remember(admin_id, idempotency_key, result)
0172         self._write_audit(request_id, task, "task_release", "success")
0173         return result
0174
0175     def decide(
0176         self,
0177         task_id: str,
0178         *,
0179         object_version: int,
0180         admin_id: str,
0181         request_id: str,
0182         idempotency_key: str,
0183         action: str,
0184         action_code: str | None = None,
0185         fact_note: str | None = None,
0186         due_at: datetime | None = None,
0187         internal_reason: str | None = None,
0188     ) -> TaskMutationResult:
0189         task = self._get_mutable(task_id, object_version, admin_id, idempotency_key)
0190         if task.get("assigned_admin_id") != admin_id or task["state"] != "claimed":
0191             raise ApiException(403, "FORBIDDEN")
0192         allowed = set(self._allowed_actions(task["task_kind"]))
0193         if action not in allowed:
0194             raise ApiException(422, "VALIDATION_FAILED")
0195         if (
0196             task["task_kind"] == "followup"
0197             and action_code == "contact_made"
0198             and self.settings.environment_kind.value == "demo"
0199         ):
0200             raise ApiException(422, "VALIDATION_FAILED")
0201         task.update(
0202             {
0203                 "state": "waiting_other" if action == "safety_review" else "completed",
0204                 "version": task["version"] + 1,
0205                 "updated_at": datetime.now(UTC),
0206             }
0207         )
0208         if fact_note or internal_reason or due_at:
0209             note = fact_note or internal_reason or ""
0210             task.setdefault("records", []).append(
0211                 {
0212                     "label": "已记录事实",
```

backend/app/services/admin_workbench_service.py

```
0213         "value": note,
0214     }
0215 )
0216     if due_at:
0217         task["records"].append({"label": "下一次跟进时间", "value": due_at.isoformat()})
0218     result = self._result(task, request_id)
0219     self._remember(admin_id, idempotency_key, result)
0220     self._write_audit(request_id, task, action, "success")
0221     return result
0222
0223 def list_audit(self) -> list[AuditEvent]:
0224     return [
0225         AuditEvent(
0226             request_id=event.request_id,
0227             actor=event.actor_id,
0228             capability=event.capability or "",
0229             resource=f"{event.resource_type}:{event.resource_id}",
0230             action=event.action,
0231             data_scope=[part for part in event.data_scope.split(",") if part],
0232             outcome=cast(
0233                 Literal["success", "denied", "conflict", "failure"],
0234                 event.outcome
0235                 if event.outcome in {"success", "denied", "conflict", "failure"}
0236                 else "failure",
0237             ),
0238             reason_code=event.reason_code,
0239             occurred_at=event.occurred_at,
0240             environment_kind=self.settings.environment_kind.value,
0241         )
0242         for event in self.audit.repository.list()
0243     ]
0244
0245 def list_audit_page(
0246     self,
0247     *,
0248     from_at: datetime | None = None,
0249     to_at: datetime | None = None,
0250     resource_type: str | None = None,
0251     action: str | None = None,
0252     outcome: str | None = None,
0253     cursor: str | None = None,
0254     limit: int = 50,
0255 ) -> tuple[list[AuditEvent], str | None]:
0256     """Return a bounded, read-only audit projection with opaque offset cursors."""
0257     events = self.list_audit()
0258     filtered = [
0259         event
0260         for event in events
0261         if (from_at is None or event.occurred_at >= from_at)
0262         and (to_at is None or event.occurred_at <= to_at)
0263         and (not resource_type or event.resource.startswith(f"{resource_type}:"))
0264         and (not action or event.action == action)
0265         and (not outcome or event.outcome == outcome)
0266     ]
```

backend/app/services/admin_workbench_service.py

```
0267         ]
0268         filtered.sort(key=lambda event: event.occurred_at, reverse=True)
0269         offset = self._decode_cursor(cursor)
0270         page_size = max(1, min(limit, 100))
0271         page = filtered[offset : offset + page_size]
0272         next_cursor = str(offset + len(page)) if offset + len(page) < len(filtered) else None
0273         return page, next_cursor
0274
0275     def reset_demo(
0276         self, *, request_id: str, admin_id: str, scopes: list[str]
0277     ) -> list[dict[str, str]]:
0278         if not self.settings.demo_reset_allowed or self.settings.environment_kind.value !=
0279         "demo":
0280             self.audit.write(
0281                 request_id=request_id,
0282                 actor_type="admin",
0283                 actor_id=admin_id,
0284                 capability="super_admin",
0285                 action="demo_reset",
0286                 resource_type="demo",
0287                 resource_id="demo_namespace",
0288                 data_scope="demo_namespace",
0289                 outcome="denied",
0290                 reason_code="FORBIDDEN",
0291                 occurred_at=datetime.now(UTC),
0292             )
0293             raise ApiException(403, "FORBIDDEN")
0294         self.tasks.clear()
0295         self._seed_demo_tasks()
0296         self._idempotency.clear()
0297         results = [{"collection": scope, "state": "completed"} for scope in scopes]
0298         self.audit.write(
0299             request_id=request_id,
0300             actor_type="admin",
0301             actor_id=admin_id,
0302             capability="super_admin",
0303             action="demo_reset",
0304             resource_type="demo",
0305             resource_id="demo_namespace",
0306             data_scope=".".join(scopes),
0307             outcome="success",
0308             reason_code="demo_reset_confirmed",
0309             occurred_at=datetime.now(UTC),
0310         )
0311         return results
0312
0313     def _seed_demo_tasks(self) -> None:
0314         """Populate only synthetic, non-identifying tasks in the demo namespace."""
0315         if self.settings.environment_kind.value != "demo":
0316             return
0317         now = datetime.now(UTC)
0318         templates: list[tuple[str, TaskKind, str, list[dict[str, str]], str | None]] = [
0319             (
0320                 "task-demo-content-01",
```

backend/app/services/admin_workbench_service.py

```
0321         "content_review",
0322         "树洞内容待人工确认",
0323         [{"label": "来源", "value": "匿名树洞"}, {"label": "进入时间", "value": "今日"}],
0324         "这是一段已脱敏的演示内容。",
0325     ),
0326     (
0327         "task-demo-safety-01",
0328         "safety_support",
0329         "安全支持记录待处理",
0330         [
0331             {"label": "来源", "value": "安全确认流程"},
0332             {"label": "当前状态", "value": "待确认"},
0333         ],
0334         None,
0335     ),
0336     (
0337         "task-demo-identity-01",
0338         "identity_access",
0339         "身份授权申请待审批",
0340         [
0341             {"label": "申请范围", "value": "遮罩姓名、学号后四位"},
0342             {"label": "有效期", "value": "24 小时"},
0343         ],
0344         None,
0345     ),
0346     (
0347         "task-demo-followup-01",
0348         "followup",
0349         "跟进事实待记录",
0350         [
0351             {"label": "来源", "value": "演示跟进"},
0352             {"label": "当前状态", "value": "等待后续事实"},
0353         ],
0354         None,
0355     ),
0356 ]
0357 self.tasks = {
0358     task_id: {
0359         "task_id": task_id,
0360         "task_kind": task_kind,
0361         "state": "needs_action",
0362         "created_at": now - timedelta(minutes=index * 7),
0363         "updated_at": now - timedelta(minutes=index * 7),
0364         "assigned_admin_id": None,
0365         "safe_summary": safe_summary,
0366         "version": 1,
0367         "facts": facts,
0368         "records": [],
0369         "redacted_content": redacted_content,
0370         "is_deleted": False,
```

backend/app/services/admin_workbench_service.py

```
0371         }
0372         for index, (task_id, task_kind, safe_summary, facts, redacted_content) in enumerate(
0373             templates
0374         )
0375     }
0376
0377     def _get_mutable(
0378         self, task_id: str, object_version: int, admin_id: str, key: str
0379     ) -> dict[str, Any]:
0380         remembered = self._idempotency.get((admin_id, key))
0381         if remembered is not None:
0382             raise ApiException(409, "IDEMPOTENCY_CONFLICT")
0383         task = self.tasks.get(task_id)
0384         if task is None or task.get("is_deleted", False):
0385             raise ApiException(404, "NOT_FOUND")
0386         if int(task["version"]) != object_version:
0387             raise ApiException(409, "VERSION_CONFLICT", current_version=task["version"])
0388         return task
0389
0390     @staticmethod
0391     def _decode_cursor(cursor: str | None) -> int:
0392         if not cursor:
0393             return 0
0394         try:
0395             return max(0, int(cursor))
0396         except ValueError as error:
0397             raise ApiException(400, "INVALID_REQUEST") from error
0398
0399     @staticmethod
0400     def _in_section(task: dict[str, Any], section: WorkbenchSection, admin_id: str) -> bool:
0401         state = task["state"]
0402         if section == "needs_action":
0403             return state == "needs_action" or (
0404                 state == "claimed" and task.get("assigned_admin_id") == admin_id
0405             )
0406         if section == "waiting_other":
0407             return state == "waiting_other" or (
0408                 state == "claimed" and task.get("assigned_admin_id") != admin_id
0409             )
0410         if section == "recent":
0411             return state in {"completed", "cancelled"}
0412         return True
0413
0414     def _summary(self, task: dict[str, Any], *, admin_id: str) -> TaskSummary:
0415         assigned = task.get("assigned_admin_id")
0416         return TaskSummary(
0417             task_id=task["task_id"],
0418             task_kind=task["task_kind"],
0419             state=task["state"],
0420             created_at=task["created_at"],
0421             updated_at=task["updated_at"],
0422             assigned_admin_display=("当前工作人员" if assigned == admin_id else "其他工作人员")
0423             if assigned
0424             else None,
```

backend/app/services/admin_workbench_service.py

```
0425         safe_summary=task["safe_summary"],
0426         object_version=task["version"],
0427     )
0429     def _detail(self, task: dict[str, Any]) -> TaskDetail:
0430         return TaskDetail(
0431             task_id=task["task_id"],
0432             task_kind=task["task_kind"],
0433             state=task["state"],
0434             object_version=task["version"],
0435             facts=[TaskFact(**fact) for fact in task.get("facts", [])],
0436             redacted_content=task.get("redacted_content"),
0437             allowed_actions=self._allowed_actions(task["task_kind"]),
0438             records=[TaskFact(**fact) for fact in task.get("records", [])],
0439             environment_kind=self.settings.environment_kind.value,
0440         )
0442     @staticmethod
0443     def _allowed_actions(task_kind: TaskKind) -> list[str]:
0444         return {
0445             "content_review": ["publish", "protect", "unpublish", "safety_review"],
0446             "safety_support": ["record_support", "set_followup", "complete"],
0447             "identity_access": ["approve", "deny", "revoke"],
0448             "followup": ["record_followup", "complete"],
0449         }[task_kind]
0451     @staticmethod
0452     def _result(task: dict[str, Any], request_id: str) -> TaskMutationResult:
0453         return TaskMutationResult(
0454             task_id=task["task_id"],
0455             new_state=task["state"],
0456             new_object_version=task["version"],
0457             audit_request_id=request_id,
0458         )
0460     def _remember(self, admin_id: str, key: str, result: TaskMutationResult) -> None:
0461         self._idempotency[(admin_id, key)] = result
0463     def _write_audit(
0464         self, request_id: str, task: dict[str, Any], action: str, outcome: str
0465     ) -> None:
0466         self.audit.write(
0467             request_id=request_id,
0468             actor_type="admin",
0469             actor_id=str(task.get("assigned_admin_id") or "admin"),
0470             capability="super_admin",
0471             action=action,
0472             resource_type="task",
0473             resource_id=task["task_id"],
0474             data_scope="task_state,object_version",
0475             outcome=outcome,
0476             reason_code=None,
0477             occurred_at=datetime.now(UTC),
0478             facts={"task_kind": task["task_kind"], "object_version": task["version"]},
0479         )
```